

ВОПРОС-ОТВЕТ

МЕТОДИЧКА ПО JAVA

АВТОР: IVAN CHEMAKIN

ВОПРОС-ОТВЕТ

МЕТОДИЧКА ПО JAVA

АВТОР IVAN CHEMAKIN

© Иван Чемакин

Данная книга предназначена для обучения по Java разработке, подготовке к техническому интервью и помощи при дальнейшей разработке.

Иван Чемакин

work.vanyachemakin@gmail.com

<https://github.com/vanyachemakin-code>

1.

ОСНОВЫ ОСНОВ.



1. ЧТО ТАКОЕ JVM? (JAVA VIRTUAL MACHINE) – ВИРТУАЛЬНАЯ МАШИНА JAVA, КОТОРАЯ ВЫПОЛНЯЕТ JAVA-БАЙТКОД. ВСЕ ПРОГРАММЫ НА JAVA КОМПИЛИРУЮТСЯ В БАЙТКОД, КОТОРЫЙ МОЖЕТ БЫТЬ ВЫПОЛНЕН НА ЛЮБОЙ ПЛАТФОРМЕ, НА КОТОРУЮ УСТАНОВЛЕНА JVM.

ПРИЧИНЫ, ПОЧЕМУ JVM ВАЖНА И ШИРОКО ИСПОЛЬЗУЕТСЯ В РАЗРАБОТКЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ:

- **Переносимость:** код Java можно написать один раз и запустить на любой платформе, на которой установлена JVM, независимо от базового оборудования и операционной системы. Это делает Java-программы легко переносимыми и уменьшает количество кода, необходимого для конкретной платформы.
- **Управление памятью:** JVM управляет распределением памяти и автоматически освобождает неиспользуемую память посредством сборки мусора. Это освобождает разработчиков от утомительной и чреватой ошибками задачи ручного управления памятью.
- **Безопасность.** Поскольку JVM выполняет код Java в изолированной среде, это предотвращает причинение вреда базовой системе вредоносным кодом. Это делает Java популярным выбором для создания безопасных и надежных приложений.
- **Производительность:** JVM создана для оптимизации выполнения кода Java и использует передовые методы, такие как своевременная компиляция, для достижения высокой производительности.

2. ЧТО ТАКОЕ JRE (JAVA RUNTIME ENVIRONMENT)? — ЭТО СРЕДА ВЫПОЛНЕНИЯ, НАБОР ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ, НЕОБХОДИМЫЙ ДЛЯ ЗАПУСКА ПРИЛОЖЕНИЙ, НАПИСАННЫХ НА ЯЗЫКЕ ПРОГРАММИРОВАНИЯ JAVA.

КОМПОНЕНТЫ:

- **JVM (Java Virtual Machine)**, исполняет байт-код.
- Стандартный набор библиотек классов (**Java Class Library**).

JRE ИСПОЛЬЗУЕТСЯ ПОЛЬЗОВАТЕЛЯМИ **ДЛЯ ЗАПУСКА ПРОГРАММ**, А НЕ ДЛЯ ИХ РАЗРАБОТКИ.

3. ЧТО ТАКОЕ JDK (JAVA DEVELOPMENT KIT)? — ЭТО БЕСПЛАТНЫЙ КОМПЛЕКТ РАЗРАБОТЧИКА, ПРЕДОСТАВЛЯЮЩИЙ ВСЕ НЕОБХОДИМЫЕ ИНСТРУМЕНТЫ ДЛЯ СОЗДАНИЯ, КОМПИЛЯЦИИ, ОТЛАДКИ И ЗАПУСКА ПРИЛОЖЕНИЙ НА ЯЗЫКЕ JAVA.

КОМПОНЕНТЫ:

- **JVM** (виртуальную машину).
- **JRE** (среду выполнения), обеспечивает работу уже готовой программы.
- **Компилятор `javac`**, переводит исходный код `.java` в байт-код `.class`.
- **Утилиты**: инструменты для отладки, документирования (**`javadoc`**) и упаковки.
- **Библиотеки классов**: набор готовых инструментов для разработки.

**ЭТО БАЗОВЫЙ ИНСТРУМЕНТ, НЕОБХОДИМЫЙ ДЛЯ
JAVA-ПРОГРАММИРОВАНИЯ.**

4. КАК РАБОТАЕТ ПАМЯТЬ В JAVA? — ПАМЯТЬ ДЕЛИТСЯ НА ДВЕ ОСНОВНЫЕ ЧАСТИ: КУЧА (HEAP) И СТЕК (STACK).

СТЕК (STACK):

- Для каждого потока создается свой стек.
- Хранит локальные переменные (примитивы) и ссылки на объекты в куче.
- Когда метод завершается, его фрейм стека уничтожается.

МЕТАДАННЫЕ (METASPACE):

- Хранит информацию о классах, методах, аннотаций и константы.
- Хранится в нативной памяти (off-heap), очищается сборщиком мусора.

КУЧА (HEAP):

- Глобальная область для всех объектов, созданных с помощью new. Доступна всем потокам.
- Пул Строк (**String Pool**): Хранит уникальные строковые литералы, находится в куче.
- Также хранит ссылки на объекты (если ссылки объявлены внутри объекта).
- Управляется сборщиком мусора (**garbage collector**).
- Делится на:
 - ➔ **Молодое поколение (Young Generation)**: Содержит новые объекты:
 - **Eden Space**: сюда попадают все новые объекты.
 - **Survivor Spaces(S0 и S1)**: две зоны («from» и «to»), где хранятся объекты, пережившие как минимум один цикл сборки мусора в Eden.
 - **Цикл жизни объектов**: При Minor GC (малая сборка мусора) живые объекты из Eden и одного Survivor (S0) перемещаются во второй Survivor (S1), а затем роли зон меняются.
 - ➔ **Старое поколение (Old Generation)**: Объекты, пережившие несколько циклов garbage collector.

5. ЧТО ТАКОЕ CLASSLOADER (ЗАГРУЗЧИК КЛАССОВ)? — ЭТО СПЕЦИАЛЬНЫЙ КОМПОНЕНТ JRE, КОТОРЫЙ ДИНАМИЧЕСКИ ЗАГРУЖАЕТ КЛАССЫ JAVA В ОПЕРАТИВНУЮ ПАМЯТЬ JVM ВО ВРЕМЯ ВЫПОЛНЕНИЯ ПРОГРАММЫ. ВМЕСТО ЗАГРУЗКИ ВСЕХ .CLASS ФАЙЛОВ ПРИ СТАРТЕ ПРИЛОЖЕНИЯ, JAVA ЗАГРУЖАЕТ ИХ ПО МЕРЕ НЕОБХОДИМОСТИ (ПРИ ПЕРВОМ ОБРАЩЕНИИ К КЛАССУ В КОДЕ).

ТРИ ОСНОВНЫХ ТИПА CLASSLOADER:

➔ Bootstrap ClassLoader (Базовый)

- Главный загрузчик, написанный на нативном коде (C/C++).
- Не имеет родителя и скрыт от Java-кода (вызов `String.class.getClassLoader()` вернет null).
- **Загружает самое сердце Java SE (класс `Object`, строки, базовые коллекции).**

➔ Platform ClassLoader (Платформенный)

- Является потомком Bootstrap.
- **Догружает остальные, менее критичные для старта, но стандартные модули и расширения Java SE (например, библиотеки для работы с XML, криптографией или SQL).**

➔ Application ClassLoader (Системный)

- Также известен как System ClassLoader.
- Является потомком Platform загрузчика.
- **Загружает классы вашего приложения и сторонние библиотеки, прописанные в путях запуска (`-classpath` или `—module-path`).**

ЗАГРУЗЧИКИ КЛАССОВ ИСПОЛЬЗУЮТСЯ В JAVA ДЛЯ СЛЕДУЮЩИХ ЦЕЛЕЙ:

- **Разделение классов** - различные загрузчики классов могут загружать классы из разных источников и иметь свою собственную область видимости, что позволяет избежать конфликтов имен классов.
- **Динамическая загрузка классов** - загрузчики классов позволяют загружать классы во время выполнения программы, что может быть полезно при создании расширяемых приложений.
- **Изоляция кода** - загрузчики классов могут загружать классы в изолированной среде, что предотвращает несанкционированный доступ к чувствительным данным и защищает систему от ошибок в коде.

ОСНОВНЫЕ ПРИНЦИПЫ РАБОТЫ:

- **Модель делегирования (Delegation Model):** Когда ClassLoader получает запрос на поиск класса, он не ищет его сам сразу. Он передает запрос вверх своему родителю. Поиск опускается обратно к текущему загрузчику только в том случае, если ни один из вышестоящих предков не смог найти данный класс.
- **Видимость (Visibility):** Классы, загруженные родителем, видны его дочерним загрузчикам. Но дочерние классы не видны родительским загрузчикам.
- **Уникальность (Uniqueness):** Позволяет избежать дублирования классов. Если родительский загрузчик уже загрузил класс, дочерний не будет загружать его повторно.

6. ЧТО ТАКОЕ RUN-TIME DATA AREAS? — ЭТО ОБЛАСТИ ПАМЯТИ, КОТОРЫЕ ВЫДЕЛЯЮТСЯ ДЛЯ ХРАНЕНИЯ ДАННЫХ ВО ВРЕМЯ ВЫПОЛНЕНИЯ.

- **Method Area** - область памяти, которая хранит описания классов, методов и других метаданных.
- **Heap** - область памяти, которая хранит объекты, созданные во время выполнения программы.
- **Java Stack** - область памяти, которая хранит данные локальных переменных и стек вызовов для каждого потока исполнения.
- **Native Method Stack** - область памяти, которая хранит данные для вызова методов на языке, отличном от Java.
- **PC Register** - регистр, который содержит текущую инструкцию **JVM** для каждого потока исполнения.
- **Direct Memory** - область памяти, которая используется для работы с прямой буферизацией данных.

7. ЧТО ТАКОЕ СБОРЩИК МУСОРА (GARBAGE COLLECTOR)? — ЭТО ЧАСТЬ ВИРТУАЛЬНОЙ МАШИНЫ JAVA (JVM), КОТОРАЯ АВТОМАТИЧЕСКИ УПРАВЛЯЕТ ПАМЯТЬЮ, НАХОДЯ И УДАЛЯЯ ОБЪЕКТЫ, КОТОРЫЕ ПРОГРАММА БОЛЬШЕ НЕ ИСПОЛЬЗУЕТ («МУСОР»).

ОСНОВНЫЕ ПРИНЦИПЫ РАБОТЫ:

- **Автоматизация:** Создание объектов идет через `new`, а уничтожение — автоматически через `Garbage Collector`.
- **Алгоритм трассировки:** `Garbage Collector` отслеживает ссылки на объекты, удаляя те, на которые больше нет ссылок («мертвые» объекты).
- **Поколения памяти:** Большинство `Garbage Collector` делят кучу (**Heap**) на «молодое» (**Young**) и «старое» (**Old**) поколения, так как большинство объектов живут недолго.
- **Stop-the-world (STW):** Часто сборка мусора вызывает паузу в работе приложения.

8. ЧТО ТАКОЕ ПРОКСИ-ОБЪЕКТ (PROXY OBJECT) — ЭТО ОБЪЕКТ, КОТОРЫЙ ВЫСТУПАЕТ В КАЧЕСТВЕ ЗАМЕНИТЕЛЯ ДРУГОГО ОБЪЕКТА И КОНТРОЛИРУЕТ ДОСТУП К НЕМУ. ПРОКСИ-ОБЪЕКТ МОЖЕТ ИСПОЛЬЗОВАТЬСЯ ДЛЯ ПЕРЕДАЧИ ЗАПРОСОВ К ОРИГИНАЛЬНОМУ ОБЪЕКТУ ЧЕРЕЗ ПРОМЕЖУТОЧНЫЙ УРОВЕНЬ, ЧТО ПОЗВОЛЯЕТ ВЫПОЛНЯТЬ ДОПОЛНИТЕЛЬНУЮ ОБРАБОТКУ ИЛИ ПРОВЕРКУ ПЕРЕД ВЫПОЛНЕНИЕМ ЗАПРОСА.

В JAVA ПРОКСИ-ОБЪЕКТЫ СОЗДАЮТСЯ С ПОМОЩЬЮ ИНТЕРФЕЙСОВ. ЕСЛИ У НАС ЕСТЬ ИНТЕРФЕЙС, КОТОРЫЙ ОПРЕДЕЛЯЕТ МЕТОДЫ, КОТОРЫЕ ДОЛЖНЫ ВЫЗЫВАТЬСЯ НА ОРИГИНАЛЬНОМ ОБЪЕКТЕ, МЫ МОЖЕМ СОЗДАТЬ ПРОКСИ-ОБЪЕКТ, КОТОРЫЙ РЕАЛИЗУЕТ ЭТОТ ИНТЕРФЕЙС И ПЕРЕНАПРАВЛЯЕТ ВЫЗОВЫ МЕТОДОВ К ОРИГИНАЛЬНОМУ ОБЪЕКТУ. ПРИ ЭТОМ МЫ МОЖЕМ ВЫПОЛНЯТЬ НУЖНЫЕ ОПЕРАЦИИ ДО ИЛИ ПОСЛЕ ВЫЗОВА МЕТОДОВ НА ОРИГИНАЛЬНОМ ОБЪЕКТЕ.

ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ ПРОКСИ-ОБЪЕКТОВ В JAVA:

- **Кэширование данных:** если мы хотим кэшировать результаты вызовов методов на объекте, мы можем создать прокси-объект, который будет хранить результаты предыдущих вызовов и возвращать их без вызова методов на оригинальном объекте.
- **Логирование:** мы можем создать прокси-объект, который будет записывать информацию о вызовах методов на оригинальном объекте в лог-файл, чтобы отслеживать его работу.
- **Удаленный доступ:** прокси-объекты могут использоваться для организации удаленного доступа к объектам через сеть. При этом прокси-объект на клиентской стороне будет передавать запросы на вызов методов на сервер, а прокси-объект на серверной стороне уже будет вызывать методы на реальном объекте и возвращать результат клиенту.

9. ЧТО ТАКОЕ КОВАРИАНТНОСТЬ ТИПОВ? — ЭТО СВОЙСТВО НЕКОТОРЫХ ЯЗЫКОВ ПРОГРАММИРОВАНИЯ, КОТОРОЕ ПОЗВОЛЯЕТ ИСПОЛЬЗОВАТЬ ПРОИЗВОДНЫЙ ТИП ВМЕСТО БАЗОВОГО ТИПА В КОНТЕКСТЕ, ГДЕ ОЖИДАЕТСЯ БАЗОВЫЙ ТИП. ДРУГИМИ СЛОВАМИ, КОВАРИАНТНОСТЬ ПОЗВОЛЯЕТ ИСПОЛЬЗОВАТЬ ОБЪЕКТЫ ПРОИЗВОДНЫХ КЛАССОВ ТАМ, ГДЕ ТРЕБУЕТСЯ ОБЪЕКТ БАЗОВОГО КЛАССА.

В JAVA КОВАРИАНТНОСТЬ ТИПОВ ИСПОЛЬЗУЕТСЯ В ОТНОШЕНИИ **НАСЛЕДОВАНИЯ И ПЕРЕОПРЕДЕЛЕНИЯ** МЕТОДОВ. КОГДА МЕТОД В ПОДКЛАССЕ ИМЕЕТ ВОЗВРАЩАЕМЫЙ ТИП, КОТОРЫЙ ЯВЛЯЕТСЯ ПРОИЗВОДНЫМ ОТ ВОЗВРАЩАЕМОГО ТИПА МЕТОДА В СУПЕРКЛАССЕ, ТО ЭТОТ ТИП СЧИТАЕТСЯ КОВАРИАНТНЫМ.

**10. ЧТО ТАКОЕ CASTING
(ПРЕОБРАЗОВАНИЕ ТИПА)? — ЭТО
ПРОЦЕСС ПРЕОБРАЗОВАНИЯ
ЗНАЧЕНИЯ ОДНОГО ТИПА В ЗНАЧЕНИЕ
ДРУГОГО ТИПА.**

**ИСКЛЮЧЕНИЕ `CLASSCASTEXCEPTION`
ВОЗНИКАЕТ, КОГДА ПРОИСХОДИТ ПОПЫТКА
ПРИВЕСТИ ОБЪЕКТ К НЕВЕРНОМУ ТИПУ ВО ВРЕМЯ
ВЫПОЛНЕНИЯ ПРОГРАММЫ.**

В JAVA ЕСТЬ ДВА ТИПА ПРИВЕДЕНИЯ, КОТОРЫЕ МОГУТ БЫТЬ ИСПОЛЬЗОВАНЫ ДЛЯ ПРЕОБРАЗОВАНИЯ ТИПОВ:

- **Неявное приведение** выполняется автоматически компилятором, когда значения одного типа используются в контексте, где ожидается другой тип.
- **Явное приведение** выполняется с помощью оператора приведения **(type)value**. Эта операция используется, когда необходимо преобразовать значение одного типа в другой тип явным образом.

**ЧТОБЫ ИЗБЕЖАТЬ `CLASSCASTEXCEPTION`, СЛЕДУЕТ
ИСПОЛЬЗОВАТЬ ОПЕРАТОР `INSTANCEOF`, ЧТОБЫ
ПРОВЕРИТЬ ТИП ОБЪЕКТА ПЕРЕД ЕГО ПРИВЕДЕНИЕМ К
ДРУГОМУ ТИПУ.**

11. ЧТО ТАКОЕ MAVEN И GRADLE? — ЭТО ИНСТРУМЕНТЫ АВТОМАТИЗАЦИИ СБОРКИ JAVA-ПРОЕКТОВ, КОТОРЫЕ УПРАВЛЯЮТ БИБЛИОТЕКАМИ (ЗАВИСИМОСТЯМИ), КОМПИЛИРУЮТ КОД, ЗАПУСКАЮТ ТЕСТЫ И УПАКОВЫВАЮТ ПРИЛОЖЕНИЯ.

ОСНОВНЫЕ РАЗЛИЧИЯ:

➔ MAVEN (APACHE MAVEN)

- **Конфигурация:** XML (понятный, строгий, но многословный).
- **Подход:** Основан на жизненном цикле (стандартные этапы: **clean, compile, test, package**).
- **Плюсы:** Прост в освоении, стандарт для многих старых проектов.

➔ GRADLE

- **Конфигурация:** Groovy или Kotlin DSL (лаконичный, гибкий).
- **Подход:** Основан на графе задач (более гибкий для сложной логики).
- **Плюсы:** Высокая производительность, отлично подходит для больших проектов и **Android**.

12. ЧТО ТАКОЕ СЕРИАЛИЗАЦИЯ И ДЕСЕРИАЛИЗАЦИЯ?

СЕРИАЛИЗАЦИЯ — ЭТО ПРОЦЕСС ПРЕОБРАЗОВАНИЯ ОБЪЕКТА ИЗ ОПЕРАТИВНОЙ ПАМЯТИ (СТРУКТУРЫ ДАННЫХ) В ПОСЛЕДОВАТЕЛЬНОСТЬ БАЙТОВ, СТРОКУ (НАПРИМЕР, JSON, XML) ИЛИ ДРУГОЙ ФОРМАТ ДЛЯ СОХРАНЕНИЯ НА ДИСК ИЛИ ПЕРЕДАЧИ ПО СЕТИ.

ДЕСЕРИАЛИЗАЦИЯ — ЭТО ОБРАТНЫЙ ПРОЦЕСС: ВОССТАНОВЛЕНИЕ ОБЪЕКТА ИЗ ЭТОГО ПОТОКА ДАННЫХ.

13. ЧТО ТАКОЕ BIG O NOTATION? — ЭТО МАТЕМАТИЧЕСКАЯ СИСТЕМА, ИСПОЛЬЗУЕМАЯ В КОМПЬЮТЕРНЫХ НАУКАХ ДЛЯ ОЦЕНКИ ЭФФЕКТИВНОСТИ И СЛОЖНОСТИ АЛГОРИТМОВ. ОНА ПОКАЗЫВАЕТ, КАК ИЗМЕНЯЕТСЯ ВРЕМЯ ВЫПОЛНЕНИЯ ПРОГРАММЫ ИЛИ ОБЪЕМ ИСПОЛЬЗУЕМОЙ ПАМЯТИ ПРИ УВЕЛИЧЕНИИ КОЛИЧЕСТВА ВХОДНЫХ ДАННЫХ (N).

ОСНОВНЫЕ ТИПЫ СЛОЖНОСТИ (ОТ ЛУЧШИХ К ХУДШИМ):

- **$O(1)$ Константная:** Время не зависит от объема данных (n). Получение элемента массива по индексу.
- **$O(\log n)$ Логарифмическая:** Область поиска уменьшается вдвое на каждом шаге. Бинарный поиск в отсортированном массиве.
- **$O(n)$ Линейная:** Время растет прямо пропорционально размеру данных. Поиск элемента перебором (простой цикл).
- **$O(n \log n)$ Линейно-логарифмическая:** Комбинация линейного обхода и деления данных. Эффективные сортировки (Быстрая, Слиянием).
- **$O(n^2)$ Квадратичная:** Время растет как квадрат от объема входных данных. Вложенные циклы, сортировка пузырьком.
- **$O(2^n)$ Экспоненциальная:** Время удваивается с каждым новым элементом. Рекурсивный поиск чисел Фибоначчи «в лоб».
- **$O(n!)$ Факториальная:** Самый медленный тип сложности, растет взрывообразно. Задача коммивояжера полным перебором.

КЛЮЧЕВЫЕ ПРАВИЛА РАСЧЕТА BIG O:

- **Константы отбрасываются:** Алгоритм, делающий $2n$ операций, и алгоритм с $5n$ операциями имеют одинаковую линейную сложность — $O(n)$.
- **Учитывается только старшая степень:** Если функция описывается как $n^2 + n + 5$, то Big O будет равен $O(n^2)$, поскольку при больших n младшие члены почти не влияют на общую скорость.
- **Последовательные действия складываются:** Если сначала идет цикл на n элементов, а за ним еще один на n элементов, общая сложность равна $O(n) + O(n) = O(n)$.
- **Вложенные действия перемножаются:** Если один цикл на n элементов находится внутри другого цикла на n элементов, их сложности перемножаются: $O(n \times n) = O(n^2)$.

ДВА ВИДА СЛОЖНОСТИ АЛГОРИТМОВ:

- **Временная сложность (Time Complexity):** Оценивает количество процессорных операций и время, необходимое для выполнения программы.
- **Пространственная сложность (Space Complexity):** Оценивает объем дополнительной оперативной памяти, которую алгоритм выделяет в процессе своей работы.

L

ООП ЧТО ЭТО ТАКОЕ???



1. ЧТО ТАКОЕ ООП? (ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ) — ЭТО МЕТОДОЛОГИЯ ПРОГРАММИРОВАНИЯ, В КОТОРОЙ ПРОГРАММА СТРОИТСЯ НА ОСНОВЕ ОБЪЕКТОВ, КОТОРЫЕ ИМЕЮТ СВОЙСТВА И ПОВЕДЕНИЕ. ОСНОВНЫЕ ПРИНЦИПЫ ООП ВКЛЮЧАЮТ ИНКАПСУЛЯЦИЮ, НАСЛЕДОВАНИЕ И ПОЛИМОРФИЗМ.

ОСНОВНЫЕ ПРИНЦИПЫ ООП:

- **Инкапсуляция** - это принцип, который позволяет скрыть детали реализации объекта от других объектов. Таким образом, объект может предоставить только необходимый интерфейс для работы с ним. Например, класс "Человек" может иметь свойство "Возраст", но этот возраст может быть доступен только через метод получения.
- **Наследование** - это принцип, который позволяет создавать новые классы на основе уже существующих. Новый класс наследует свойства и методы родительского класса и может добавить свои собственные свойства и методы. Например, класс "Сотрудник" может наследовать свойства и методы от класса «Человек». Ключевое слово «super» позволяет обращаться к логике метода родительского класса.
- **Полиморфизм** - это принцип, который позволяет объектам с одинаковым интерфейсом иметь различную реализацию. Такой подход позволяет использовать один и тот же метод для работы с разными типами объектов. Делится на статический (перегрузка методов) и динамический (переопределение методов).
- **Абстракция** - разделение сложной системы на более простые компоненты, скрывая внутренние детали реализации и оставляя только интерфейс взаимодействия. Реализуется через абстрактные классы и интерфейсы.

2. СРАВНЕНИЕ ООП С ПРОЦЕДУРНЫМ/ ФУНКЦИОНАЛЬНЫМ ПРОГРАММИРОВАНИЕМ.

ПРЕИМУЩЕСТВА:

- **Классы и объекты** позволяют создавать более структурированный и организованный код, благодаря чему его легче читать, понимать и поддерживать.
- **Наследование** позволяет повторно использовать код и создавать новые классы на основе существующих, что упрощает разработку и поддержку приложений.
- **Инкапсуляция** позволяет скрыть детали реализации от пользователя, обеспечивая лучшую защиту данных и большую безопасность кода.
- **Полиморфизм** позволяет работать с объектами разных классов через единый интерфейс, упрощая разработку и повышая гибкость приложения.

НЕДОСТАТКИ:

- Разработка объектно-ориентированных приложений может быть **сложнее и затратнее** по времени, чем процедурная или функциональная разработка.
- В некоторых случаях ООП может привести к **избыточности кода и лишней абстракции**, что усложняет его понимание и поддержку.
- Из-за большего количества абстракций и сложности объектных структур, могут возникать проблемы с **производительностью** приложений.
- Некоторые задачи лучше решаются с помощью процедурного или функционального программирования, например, математические вычисления или обработка больших объемов данных.

3. ЧТО ТАКОЕ HIGH COHESION И LOW COUPLING? — ЭТО ДВА ПРИНЦИПА ОБЪЕКТНО-ОРИЕНТИРОВАННОГО ПРОГРАММИРОВАНИЯ, КОТОРЫЕ НАПРАВЛЕННЫ НА УЛУЧШЕНИЕ КАЧЕСТВА КОДА И ЕГО ПОДДЕРЖКИ.

ОБЩИЙ ПРИНЦИП ЗАКЛЮЧАЕТСЯ В ТОМ, ЧТО КАЖДЫЙ МОДУЛЬ ДОЛЖЕН ИМЕТЬ ТОЛЬКО **ОДНУ ОТВЕТСТВЕННОСТЬ И МИНИМАЛЬНО ЗАВИСЕТЬ ОТ ДРУГИХ МОДУЛЕЙ**, ЧТОБЫ КОД БЫЛ ЛЕГКО ЧИТАЕМЫМ, ПОНЯТНЫМ И ПОДДЕРЖИВАЕМЫМ.

➔ **High Cohesion (Высокая связность)** - каждый модуль должен иметь только одну ответственность и все его элементы должны быть тесно связаны между собой. Это означает, что каждый модуль должен быть структурирован таким образом, чтобы его элементы выполняли только свои задачи, без лишних действий и зависимостей от других модулей.

ПРИМЕР HIGH COHESION: КЛАСС ДЛЯ РАБОТЫ С БАЗОЙ ДАННЫХ ДОЛЖЕН СОДЕРЖАТЬ ТОЛЬКО МЕТОДЫ ДЛЯ РАБОТЫ С БАЗОЙ ДАННЫХ.

➔ **Low Coupling (Низкая связность)** - модули программы должны быть слабо связаны друг с другом. Это означает, что каждый модуль должен иметь минимальные зависимости от других модулей, чтобы можно было легко менять, удалять или заменять его без изменения других модулей.

ПРИМЕР LOW COUPLING: КЛАСС ДЛЯ РАБОТЫ С БАЗОЙ ДАННЫХ НЕ ДОЛЖЕН СОДЕРЖАТЬ ЗАВИСИМОСТИ ОТ ИНТЕРФЕЙСА ПОЛЬЗОВАТЕЛЯ ИЛИ ДРУГИХ МОДУЛЕЙ, ЧТОБЫ МОЖНО БЫЛО ЛЕГКО ЗАМЕНИТЬ ЕГО НА ДРУГУЮ РЕАЛИЗАЦИЮ БАЗЫ ДАННЫХ.

4. ЧТО ТАКОЕ АССОЦИАТИВНЫЕ СВЯЗИ МЕЖДУ ОБЪЕКТАМИ? — ЭТО ОТНОШЕНИЯ МЕЖДУ ОБЪЕКТАМИ ДВУХ КЛАССОВ, ПРИ КОТОРЫХ ОНИ ССЫЛАЮТСЯ ДРУГ НА ДРУГА, ПОЗВОЛЯЯ ИСПОЛЬЗОВАТЬ ФУНКЦИОНАЛЬНОСТЬ ДРУГ ДРУГА.

СУЩЕСТВУЕТ НЕСКОЛЬКО ВИДОВ АССОЦИАТИВНЫХ СВЯЗЕЙ МЕЖДУ ОБЪЕКТАМИ:

- ▶ **Агрегация** - это отношение целое-часть, где один объект является "контейнером" для другого объекта, и включает его в свой состав. Объекты могут существовать независимо друг от друга.
- ▶ **Композиция** - это также отношение целое-часть, но здесь объекты жестко связаны друг с другом, при этом родительский объект создает и управляет жизненным циклом дочернего объекта. Если родительский объект уничтожается, то дочерний объект также уничтожается.
- ▶ **Ассоциация** - это обобщенное отношение между двумя объектами, которые могут взаимодействовать друг с другом. Один объект может иметь ссылку на другой объект, но это не означает, что они являются частями друг друга или зависят друг от друга.
- ▶ **Наследование** - это отношение, при котором класс наследует свойства и методы другого класса (родительского класса). Это позволяет создавать более специализированные версии классов на основе базовых классов.
- ▶ **Реализация** - это отношение, при котором класс реализует (или выполняет) методы интерфейса. Это позволяет использовать объекты различных классов с единым интерфейсом.

5. В ЧЕМ РАЗНИЦА МЕЖДУ СТАТИЧЕСКИМ И ДИНАМИЧЕСКИМ СВЯЗЫВАНИЕМ JAVA?

- **Статическое связывание происходит во время компиляции кода** и определяет, какой метод будет вызван на основе типа переменной или ссылки на объект, которая содержит метод. Если тип переменной или ссылки заранее известен, то компилятор может точно определить, какой метод будет вызван, и связать его с этой переменной или ссылкой.
- **Динамическое связывание происходит во время выполнения программы** и определяет, какой метод будет вызван на основе фактического типа объекта, на который ссылается переменная или ссылка. Если тип объекта не известен заранее, то компилятор не может точно определить, какой метод будет вызван, и связь происходит только во время выполнения программы.

**6. ЧТО ТАКОЕ GOF ПАТТЕРНЫ? — ЭТО
23 КЛАССИЧЕСКИХ ОБЪЕКТНО-
ОРИЕНТИРОВАННЫХ ПАТТЕРНА
ПРОЕКТИРОВАНИЯ, РАЗДЕЛЕННЫХ НА
ПОРОЖДАЮЩИЕ, СТРУКТУРНЫЕ И
ПОВЕДЕНЧЕСКИЕ, КОТОРЫЕ РЕШАЮТ
ТИПИЧНЫЕ ПРОБЛЕМЫ
ПРОЕКТИРОВАНИЯ.**

- ▶ **Паттерн "Фабричный метод" (Factory Method)** - использовался для создания объектов определенного типа, в зависимости от параметров. Например, если требуется создать экземпляр класса, который может иметь различные реализации, то фабричный метод обеспечивает гибкость и удобство при создании объектов.
- ▶ **Паттерн "Абстрактная фабрика" (Abstract Factory)** - использовался для создания семейств связанных объектов. Например, если требуется создать объекты, которые зависят друг от друга и должны быть созданы вместе, то абстрактная фабрика предоставляет механизм для этого.
- ▶ **Паттерн "Одиночка" (Singleton)** - использовался для создания объекта, который может быть создан только один раз. Например, если требуется создать объект, который используется множество раз в приложении, то с помощью паттерна Одиночка можно гарантировать, что он будет создан только один раз.
- ▶ **Паттерн "Стратегия" (Strategy)** - использовался для определения алгоритма, который может быть заменен на другой алгоритм без изменения интерфейса. Например, если требуется реализовать алгоритм сортировки, то можно использовать паттерн Стратегия для того, чтобы выбирать различные методы сортировки в зависимости от конкретных требований.
- ▶ **Паттерн "Наблюдатель" (Observer)** - использовался для создания механизма, который позволяет объектам-наблюдателям получать оповещения об изменении состояния других объектов. Например, если требуется создать систему, которая обрабатывает события, то паттерн Наблюдатель может быть использован для того, чтобы отправлять уведомления о событиях всем заинтересованным объектам.

- ▶ **Паттерн "Декоратор" (Decorator)** - использовался для динамического добавления функциональности к объекту без изменения его класса. Например, если требуется добавить дополнительное поведение к объекту, то можно использовать паттерн Декоратор, который позволяет обернуть объект в другой объект с дополнительным поведением.
- ▶ **Паттерн "Адаптер" (Adapter)** - использовался для преобразования интерфейса одного класса в интерфейс другого класса. Например, если имеется класс с неподходящим интерфейсом для использования в приложении, то можно создать адаптер, который преобразует интерфейс класса в нужный интерфейс.
- ▶ **Паттерн "Итератор" (Iterator)** - использовался для последовательного доступа к элементам коллекции без раскрытия ее внутреннего представления. Например, если требуется перебрать элементы коллекции в порядке их добавления, то можно использовать паттерн Итератор, который предоставляет методы для доступа к элементам коллекции.
- ▶ **Паттерн "Шаблонный метод" (Template Method)** - использовался для определения основных шагов алгоритма, оставляя подклассам возможность переопределения некоторых шагов. Например, если требуется реализовать алгоритм, который имеет схожие шаги, но различную реализацию для каждого шага, то можно использовать паттерн Шаблонный метод, чтобы предоставить базовую реализацию алгоритма и дать возможность подклассам переопределять отдельные шаги.

- **Паттерн "Фасад" (Facade)** - использовался для предоставления упрощенного интерфейса для сложной системы. Например, если имеется сложная система, которая состоит из многих классов и компонентов, то можно создать фасад, который скрывает сложность системы и предоставляет простой интерфейс для взаимодействия с ней.
- **Паттерн "Компоновщик" (Composite)** - использовался для создания иерархических древовидных структур объектов, которые могут быть обработаны единообразно. Например, если требуется представить структуру файловой системы, то можно использовать паттерн Компоновщик для создания древовидной структуры, где папки и файлы являются узлами дерева.
- **Паттерн "Прототип" (Prototype)** - использовался для создания новых объектов путем клонирования существующих объектов. Например, если требуется создать множество объектов с одинаковыми свойствами, то можно использовать паттерн Прототип, чтобы создать первоначальный объект и затем клонировать его для создания остальных объектов.
- **Паттерн "Цепочка обязанностей" (Chain of Responsibility)** - использовался для построения цепочки объектов, которые могут обрабатывать запросы последовательно до тех пор, пока один из объектов не обработает запрос. Например, если имеется система обработки запросов и каждый запрос может быть обработан несколькими объектами, то можно использовать паттерн Цепочка обязанностей, чтобы создать цепочку объектов, которые будут обрабатывать запросы последовательно.

- **Паттерн "Состояние" (State)** - использовался для изменения поведения объекта в зависимости от его состояния. Например, если имеется объект, который может находиться в различных состояниях, то можно использовать паттерн Состояние, чтобы определить различное поведение объекта в зависимости от его текущего состояния.
- **Паттерн "Посетитель" (Visitor)** - использовался для добавления новых операций к классам, не изменяя их исходного кода. Например, если имеется множество классов и требуется добавить новую операцию, которая будет выполняться для каждого класса, то можно использовать паттерн Посетитель, чтобы добавить эту операцию без изменения исходного кода классов.
- **Паттерн "Мост" (Bridge)** - использовался для разделения абстракции и реализации, чтобы они могли изменяться независимо друг от друга. Например, если имеется класс, который представляет графический объект, то можно использовать паттерн Мост, чтобы разделить абстракцию графического объекта и его реализацию.
- **Паттерн "Легковес" (Flyweight)** - использовался для оптимизации работы с большим количеством мелких объектов, которые могут быть разделены на общие и уникальные части. Например, если требуется работать с большим количеством объектов, каждый из которых имеет много общих идентификаторов, то можно использовать паттерн Легковес, чтобы разделить общие и уникальные части объектов и оптимизировать использование памяти.

- **Паттерн "Прокси" (Proxy)** - использовался для создания объекта-заместителя, который может контролировать доступ к другому объекту. Например, если имеется объект, к которому нужно предоставить доступ только определенным пользователям, то можно использовать паттерн Прокси, который будет контролировать доступ к этому объекту.
- **Паттерн "Команда" (Command)** - использовался для инкапсуляции запроса в виде объекта, что позволяет отделить источник запроса от его исполнения. Например, если требуется реализовать систему, которая обрабатывает запросы, то можно использовать паттерн Команда, чтобы инкапсулировать запрос в виде объекта и передавать его на обработку.
- **Паттерн "Интерпретатор" (Interpreter)** - использовался для определения грамматики языка и создания интерпретатора для выполнения заданных операций. Например, если имеется язык, который нужно интерпретировать, то можно использовать паттерн Интерпретатор, который предоставляет механизм для описания грамматики языка и выполнения заданных операций.
- **Паттерн "Снимок" (Memento)** - использовался для сохранения состояния объекта и его восстановления в будущем. Например, если требуется сохранить состояние объекта перед выполнением каких-то действий, то можно использовать паттерн Снимок, чтобы сохранить его состояние и восстановить его в будущем.

- **Паттерн "Строитель" (Builder)** - использовался для создания сложных объектов путем последовательного добавления их компонентов. Например, если требуется создать объекты, которые имеют много параметров и зависят друг от друга, то можно использовать паттерн Строитель, который позволяет последовательно добавлять компоненты объекта.
- **Паттерн "Инкапсуляция состояния" (Encapsulated State)** - использовался для инкапсуляции изменений состояния объекта в соответствующие классы. Например, если требуется реализовать систему, которая обрабатывает изменения состояний объектов, то можно использовать паттерн Инкапсуляция состояния, который позволяет инкапсулировать изменения состояния объекта в соответствующие классы.
- **Паттерн "Соблюдение интерфейса" (Interface Compliance)** - использовался для создания классов, которые соответствуют определенному интерфейсу. Например, если требуется реализовать систему, которая работает с объектами, то можно использовать паттерн Соблюдение интерфейса, который обеспечивает соответствие класса заданному интерфейсу.
- **Паттерн "Реестр" (Registry)** - использовался для хранения ссылок на объекты в централизованном месте. Например, если требуется иметь доступ к объектам из разных частей приложения, то можно использовать паттерн Реестр, который позволяет хранить ссылки на объекты в централизованном месте и давать доступ к ним из разных частей приложения.

7. ЧТО ТАКОЕ SOLID? — ЭТО ПРИНЦИПЫ РАЗРАБОТКИ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ, СЛЕДУЯ КОТОРЫМ ВЫ ПОЛУЧИТЕ ХОРОШИЙ КОД, КОТОРЫЙ В ДАЛЬНЕЙШЕМ БУДЕТ ХОРОШО МАСШТАБИРОВАТЬСЯ И ПОДДЕРЖИВАТЬСЯ В РАБОЧЕМ СОСТОЯНИИ.

Single Responsibility Principle - принцип единственной ответственности. Каждый класс должен иметь только одну зону ответственности.

Open-closed Principle - принцип открытости-закрытости. Классы должны быть открыты для расширения, но закрыты для изменения.

Liskov substitution Principle - принцип подстановки Барбары Лисков. Должна быть возможность вместо базового (родительского) типа (класса) подставить любой его подтип (класс-наследник), при этом работа программы не должна измениться.

Interface Segregation Principle - принцип разделения интерфейсов. Данный принцип обозначает, что не нужно заставлять клиента (класс) реализовывать интерфейс, который не имеет к нему отношения.

Dependency Inversion Principle - принцип инверсии зависимостей. Модули верхнего уровня не должны зависеть от модулей нижнего уровня. И те, и другие должны зависеть от абстракции. Абстракции не должны зависеть от деталей. Детали должны зависеть от абстракций.

VIII.

КЛАССЫ И ОБЪЕКТЫ.



1. ЧТО ТАКОЕ CLASS OBJECT? — ЭТО ОБЪЕКТ, КОТОРЫЙ ПРЕДСТАВЛЯЕТ СОБОЙ МЕТАДАННЫЕ КЛАССА. ТО ЕСТЬ ОН СОДЕРЖИТ ИНФОРМАЦИЮ О ТОМ, КАКИМ ОБРАЗОМ БЫЛ ОПРЕДЕЛЕН КЛАСС, КАКИЕ ПОЛЯ И МЕТОДЫ ОН СОДЕРЖИТ, А ТАКЖЕ ДРУГИЕ ДАННЫЕ, НЕОБХОДИМЫЕ ДЛЯ РАБОТЫ ПРОГРАММЫ С ЭТИМ КЛАССОМ ВО ВРЕМЯ ВЫПОЛНЕНИЯ. ВСЕ ОБЪЕКТЫ УНАСЛЕДОВАНЫ ОТ НЕГО.

ОСНОВНЫЕ АСПЕКТЫ CLASS OBJECT:

- **Корневой класс:** Все классы и массивы являются наследниками **Object**.
- **Универсальность:** Ссылочная переменная типа **Object** может ссылаться на объект любого типа.

2. КАКИЕ ЕСТЬ ВИДЫ КЛАССОВ?

- **Обычные классы:** Определение базового типа, описывающее структуру, атрибуты и поведение.
- **Абстрактные классы (abstract):** Классы, которые не могут иметь экземпляров, используются как база для наследования.
- **Финальные классы (final):** Классы, которые нельзя наследовать.
- **Интерфейсы (interface):** Используются для реализации множественного наследования типов, содержат методы, которые должны быть реализованы классами.
- **Вложенные классы (nested):** Классы которые определяются внутри других классов или методов.
- **Перечисления (enum):** Особый тип класса, представляющий набор констант.
- **Записи (record):** (начиная с Java 14) Компактный способ создания классов-данных (DTO), содержащих только **поля, конструктор, геттеры, equals/hashCode/toString**.

3. ЧТО ТАКОЕ КЛАСС, ОБЪЕКТ, ИНТЕРФЕЙС?

- **Класс** - это шаблон, определяющий состояние и поведение объектов. Он содержит переменные экземпляра (состояние) и методы (поведение), которые определяют, что объекты могут делать.
- **Объект** - это экземпляр класса. Когда вы создаете объект, он получает свою собственную копию переменных экземпляра класса. Вы можете вызывать методы класса на этом объекте, чтобы изменить его состояние или получить информацию из него.
- **Интерфейс** - это контракт, который гарантирует, что класс, который реализует интерфейс, будет иметь определенные методы. Он определяет только имена методов, а не их реализацию. Класс должен реализовать все методы интерфейса, чтобы соответствовать контракту.

4. ЧТО ТАКОЕ КОНСТРУКТОР? — ЭТО МЕТОД, КОТОРЫЙ ВЫЗЫВАЕТСЯ ПРИ СОЗДАНИИ НОВОГО ОБЪЕКТА КЛАССА. КОНСТРУКТОРЫ ИСПОЛЬЗУЮТСЯ ДЛЯ ИНИЦИАЛИЗАЦИИ ОБЪЕКТОВ.

5. ЧТО ТАКОЕ IMMUTABLE OBJECT (НЕИЗМЕНЯЕМЫЙ ОБЪЕКТ)? — ЭТО ОБЪЕКТ, ЧЬЕ СОСТОЯНИЕ НЕ МОЖЕТ БЫТЬ ИЗМЕНЕНО ПОСЛЕ СОЗДАНИЯ. В JAVA НЕИЗМЕНЯЕМЫЕ ОБЪЕКТЫ ОБЫЧНО РЕАЛИЗУЮТСЯ ПУТЕМ ОБЪЯВЛЕНИЯ КЛАССА С FINAL МОДИФИКАТОРОМ И УСТАНОВКОЙ ВСЕХ ПОЛЕЙ КЛАССА КАК FINAL.

6. КАКИЕ ЕСТЬ МОДИФИКАТОРЫ ДОСТУПА?

- **Private** - ограничивает доступ к членам класса только внутри самого класса. Другие классы не могут получить доступ к приватным членам.
- **Protected** - предоставляет доступ к членам класса внутри самого класса, а также дочерним классам. Члены с модификатором `protected` также могут быть доступны и для классов из того же пакета.
- **Package-private (также называемый default)** - предоставляет доступ к членам класса только внутри того же пакета. Это является наиболее ограничительным уровнем доступа в Java
- **Public** - предоставляет доступ к членам класса из любого места программы, включая другие классы и пакеты.

МОДИФИКАТОРЫ ДОСТУПА ИСПОЛЬЗУЮТСЯ ДЛЯ ОБЕСПЕЧЕНИЯ БЕЗОПАСНОСТИ И КОНТРОЛЯ ДОСТУПА К КЛАССАМ, ПЕРЕМЕННЫМ И МЕТОДАМ. ОНИ ТАКЖЕ ПОМОГАЮТ ИЗБЕЖАТЬ ОШИБОК И КОНФЛИКТОВ ИМЁН.

7. ЧТО ТАКОЕ АБСТРАКТНЫЙ (ABSTRACT) КЛАСС? — ЭТО КЛАСС, КОТОРЫЙ НЕ МОЖЕТ БЫТЬ СОЗДАН ЭКЗЕМПЛЯРОМ. ОН СЛУЖИТ ТОЛЬКО ДЛЯ ОПИСАНИЯ ИНТЕРФЕЙСА ДЛЯ КЛАССОВ-НАСЛЕДНИКОВ. ОН СОДЕРЖИТ ХОТЯ БЫ ОДИН АБСТРАКТНЫЙ МЕТОД, КОТОРЫЙ ДОЛЖЕН БЫТЬ РЕАЛИЗОВАН В КАЖДОМ КЛАССЕ-НАСЛЕДНИКЕ. МОГУТ СОДЕРЖАТЬ ОБЫЧНЫЕ МЕТОДЫ С КОНКРЕТНОЙ РЕАЛИЗАЦИЕЙ.

КЛЮЧЕВЫЕ ОТЛИЧИЯ МЕЖДУ АБСТРАКТНЫМ КЛАССОМ И ИНТЕРФЕЙСОМ:

- **Реализация методов:** Абстрактные классы могут содержать как абстрактные, так и конкретные методы, тогда как интерфейсы могут содержать только абстрактные методы (без реализации). Также, начиная с версии Java 8, интерфейсы могут иметь реализацию методов по умолчанию (**default methods**).
- **Наследование:** Класс может наследоваться только от одного абстрактного класса, но он может реализовывать несколько интерфейсов.
- **Использование:** Абстрактные классы обычно используются там, где у нас есть общие атрибуты и поведение для группы классов, а интерфейсы используются там, где мы хотим обеспечить общую функциональность для разных классов без привязки к их иерархии наследования.
- **Наличие конструктора:** Абстрактные классы могут иметь конструкторы, тогда как интерфейсы не могут иметь конструкторов.
- **Модификаторы доступа:** Абстрактные классы могут иметь модификаторы доступа (**public, protected, private и default**), тогда как методы интерфейса по умолчанию являются **public**, а переменные интерфейса - **public static final**.

8. ЧТО ТАКОЕ ВЛОЖЕННЫЙ (NESTED) КЛАСС? — ЭТО КЛАСС, КОТОРЫЙ ОПРЕДЕЛЕН ВНУТРИ ДРУГОГО КЛАССА ИЛИ МЕТОДА. ОН МОЖЕТ БЫТЬ ОБЪЯВЛЕН КАК СТАТИЧЕСКИЙ ИЛИ НЕСТАТИЧЕСКИЙ, И МОЖЕТ ИМЕТЬ РАЗЛИЧНЫЕ УРОВНИ ДОСТУПА (PUBLIC, PRIVATE, PROTECTED).

**ОН ИСПОЛЬЗУЕТСЯ ДЛЯ
ГРУППИРОВКИ СВЯЗАННЫХ КЛАССОВ
ВМЕСТЕ И ОБЛЕГЧЕНИЯ ДОСТУПА К
НИМ ДРУГ ДРУГУ. ВЛОЖЕННЫЕ
КЛАССЫ МОГУТ ИСПОЛЬЗОВАТЬСЯ
ДЛЯ РЕАЛИЗАЦИИ СЛОЖНЫХ
АЛГОРИТМОВ, ДЛЯ ПРЕДСТАВЛЕНИЯ
КОМПОНЕНТОВ ПОЛЬЗОВАТЕЛЬСКОГО
ИНТЕРФЕЙСА, ДЛЯ СОЗДАНИЯ
ЛОГИЧЕСКИ СВЯЗАННЫХ КЛАССОВ-
ОБЕРТОК И Т.Д.**

В JAVA ЕСТЬ 4 ТИПА ВЛОЖЕННЫХ КЛАССОВ:

- **Nested Inner Class (внутренний вложенный класс)** - это нестатический вложенный класс, который определен внутри другого класса. Он имеет доступ ко всем полям и методам внешнего класса, а также может иметь свои собственные поля и методы.
- **Static Nested Class (статический вложенный класс)** - это вложенный класс, который объявлен со словом ключевым `static`. Он не имеет доступа к нестатическим полям и методам внешнего класса, но может иметь собственные статические поля и методы.
- **Local Inner Class (локальный внутренний класс)** - это вложенный класс, который определен внутри метода. Он имеет доступ к переменным и параметрам метода, а также может иметь доступ к нестатическим полям и методам внешнего класса.
- **Anonymous Inner Class (анонимный внутренний класс)** - это класс без имени, который создается непосредственно в месте использования. Он обычно используется для реализации интерфейсов или классов-абстракций "на лету" без необходимости определения нового класса.

9. ЧТО ТАКОЕ ENUM? — ЭТО ПЕРЕЧИСЛИМЫЙ ТИП ДАННЫХ, КОТОРЫЙ ПРЕДСТАВЛЯЕТ СОБОЙ НАБОР ИМЕНОВАННЫХ КОНСТАНТ. КАЖДАЯ КОНСТАНТА ПРЕДСТАВЛЯЕТ ОПРЕДЕЛЕННОЕ ЗНАЧЕНИЕ ИЗ ЗАДАННОГО СПИСКА ЗНАЧЕНИЙ. С ПОМОЩЬЮ ENUM МОЖНО СОЗДАВАТЬ КОЛЛЕКЦИИ КОНСТАНТ, КОТОРЫЕ МОГУТ ИСПОЛЬЗОВАТЬСЯ В КАЧЕСТВЕ АРГУМЕНТОВ ДЛЯ МЕТОДОВ ИЛИ СВОЙСТВ ОБЪЕКТОВ.

ВОЗМОЖНОСТИ ENUM:

- Enum может реализовывать (**implement**) интерфейс. Это означает, что каждая константа Enum будет иметь реализацию методов этого интерфейса.
- Также в Enum мы можем определять свои собственные методы, которые будут доступны для всех констант.
- Enum не может расширять (**extends**) классы, так как он уже является конечной реализацией класса.
- Можно создать **enum** без экземпляров объектов. Для этого нужно создать пустой список аргументов в определении перечисления.

МОЖЕТ ИСПОЛЬЗОВАТЬСЯ ДЛЯ СОЗДАНИЕ УТИЛИТНЫХ КЛАССОВ (UTILITY CLASSES).

- **Запрет на создание объектов:** Обычный класс для утилит требует приватного конструктора, чтобы предотвратить инстанцирование. В `enum` конструктор по умолчанию приватный, и создать экземпляр через `new` невозможно на уровне языка.
- **Запрет на наследование:** Все `enum` неявно являются `final`, поэтому от них нельзя наследоваться.
- **Безопасность и чистота кода:** Это считается более лаконичным и надежным способом реализации утилитного класса, который гарантированно не будет иметь объектов даже через **рефлексию** или **десериализацию**.
- **Покрытие тестами:** Инструменты анализа кода (например, **JaCoCo**) иногда требуют покрытия приватного конструктора в обычных классах тестами. В случае с `enum` такой проблемы нет, так как конструктор не учитывается в метриках покрытия.

10. ЧТО ДЕЛАЕТ МЕТОД ORDINAL() В ENUM? — ВОЗВРАЩАЕТ ПОРЯДКОВЫЙ НОМЕР КОНСТАНТЫ ПЕРЕЧИСЛЕНИЯ (ENUM), НАЧИНАЯ С НУЛЯ.

ПОРЯДКОВЫЙ НОМЕР — ЭТО ПОЗИЦИЯ ЭЛЕМЕНТА ПЕРЕЧИСЛЕНИЯ В СПИСКЕ ЗНАЧЕНИЙ ЭТОГО ПЕРЕЧИСЛЕНИЯ.

11. ЧТО ТАКОЕ ПРИМИТИВНЫЕ И WRAPPER (ОБЕРТКА) КЛАССЫ?

ПРИМИТИВНЫЕ КЛАССЫ — ЭТО ВСТРОЕННЫЕ ТИПЫ ДАННЫХ, КОТОРЫЕ НЕ ЯВЛЯЮТСЯ ОБЪЕКТАМИ И ИМЕЮТ ФИКСИРОВАННЫЙ РАЗМЕР.

КЛАССЫ-ОБЕРТКИ (WRAPPER CLASSES) — ЭТО СПЕЦИАЛЬНЫЕ КЛАССЫ, КОТОРЫЕ ПОЗВОЛЯЮТ РАБОТАТЬ С ПРИМИТИВНЫМИ ТИПАМИ ДАННЫХ КАК С ОБЪЕКТАМИ. ТАКИЕ КЛАССЫ ПРЕДСТАВЛЕНЫ В СТАНДАРТНОЙ БИБЛИОТЕКЕ JAVA И ИСПОЛЬЗУЮТСЯ ДЛЯ ТРАНСФОРМАЦИИ ЗНАЧЕНИЙ ПРИМИТИВНЫХ ТИПОВ ДАННЫХ В ОБЪЕКТЫ И ОБРАТНО.

ОСНОВНЫЕ РАЗЛИЧИЯ:

- **Хранение и доступ:** Примитивы хранятся в стеке (быстро), объекты-обертки — в куче (медленнее, требуют больше памяти).
- **Значение по умолчанию:** Примитивы имеют предустановленные значения (например, 0 или false), в то время как обертки — **null**, что позволяет обрабатывать отсутствие данных.
- **Функциональность:** Классы-обертки содержат полезные методы, такие как **parseInt()** или преобразование типов, а примитивы — нет.
- **Использование:** Обертки необходимы для работы с Java Collections Framework (**List**, **Map**) и **Generics**, так как коллекции принимают только объекты.
- **Сравнение:** Примитивы сравниваются через **==** (по значению), обертки — через **.equals()** (по содержимому), хотя автоупаковка может вызывать путаницу.

ВИДЫ ПЕРЕМЕННЫХ:

- ➔ **LOCAL VARIABLE** — ЭТО ПЕРЕМЕННАЯ, КОТОРАЯ ОПРЕДЕЛЕНА ВНУТРИ МЕТОДА И СУЩЕСТВУЕТ ВПЛОТЬ ДО ТОГО МОМЕНТА, ПОКА ВЫПОЛНЯЕТСЯ ЭТОТ МЕТОД.
- ➔ **INSTANCE VARIABLE** — ПЕРЕМЕННАЯ, КОТОРАЯ ОПРЕДЕЛЕНА ВНУТРИ КЛАССА, И ОНА СУЩЕСТВУЕТ ВПЛОТЬ ДО ТОГО МОМЕНТА, ПОКА СУЩЕСТВУЕТ ОБЪЕКТ.

IX.

МЕТОДЫ.



1. ЧТО ТАКОЕ ПЕРЕГРУЗКА (OVERLOADING) МЕТОДА? — ЭТО ВОЗМОЖНОСТЬ ОПРЕДЕЛЕНИЯ НЕСКОЛЬКИХ МЕТОДОВ С ОДНИМ И ТЕМ ЖЕ ИМЕНЕМ, НО С РАЗНЫМИ ПАРАМЕТРАМИ. КОМПИЛЯТОР ОПРЕДЕЛЯЕТ, КАКОЙ ИЗ ПЕРЕГРУЖЕННЫХ МЕТОДОВ НУЖНО ВЫЗВАТЬ НА ОСНОВЕ ТИПОВ АРГУМЕНТОВ, ПЕРЕДАННЫХ В ВЫЗОВЕ.

ПРИ ОПРЕДЕЛЕНИИ ПЕРЕГРУЖЕННЫХ МЕТОДОВ ВАЖНО УЧИТЫВАТЬ СЛЕДУЮЩИЕ ПРАВИЛА:

- Имена методов должны быть одинаковыми.
- Число и тип параметров должны отличаться.
- Тип возвращаемого значения может отличаться, но это не является обязательным условием.

**2. ЧТО ТАКОЕ ПЕРЕОПРЕДЕЛЕНИЕ
(OVERRIDE) МЕТОДА? — ЭТО
ВОЗМОЖНОСТЬ ИЗМЕНИТЬ
ПОВЕДЕНИЕ УНАСЛЕДОВАННОГО
МЕТОДА БЕЗ НЕОБХОДИМОСТИ
ИЗМЕНЯТЬ ЕГО ИМЯ ИЛИ СИГНАТУРУ.**

2. ЧТО ТАКОЕ СИГНАТУРА МЕТОДА? —
ЭТО УНИКАЛЬНЫЙ ИДЕНТИФИКАТОР
МЕТОДА, СОСТОЯЩИЙ ИЗ ЕГО ИМЕНИ
И УПОРЯДОЧЕННОГО СПИСКА ТИПОВ
ПАРАМЕТРОВ. ОНА ПОЗВОЛЯЕТ
КОМПИЛЯТОРУ РАЗЛИЧАТЬ МЕТОДЫ
(ПЕРЕГРУЗКА).

СИГНАТУРА НЕ ВКЛЮЧАЕТ ВОЗВРАЩАЕМЫЙ ТИП,
МОДИФИКАТОРЫ ДОСТУПА ИЛИ СПИСОК
ВЫБРАСЫВАЕМЫХ ИСКЛЮЧЕНИЙ.

ОСНОВНЫЕ КОМПОНЕНТЫ СИГНАТУРЫ:

- **Имя метода:** Например, calculate.
- **Типы параметров:** Порядок и типы. Например, (int, String).

ПОЧЕМУ ЭТО ВАЖНО:

- **Перегрузка методов:** Можно создавать методы с одинаковым именем, если их сигнатуры отличаются (разные типы или количество параметров).
- **Ошибки:** Методы с одинаковыми именами и параметрами, но разными возвращаемыми типами, вызовут ошибку компиляции.

3. ЧТО ТАКОЕ ИНИЦИАЛИЗАТОР? —
ЭТО БЛОК КОДА ВНУТРИ КЛАССА,
КОТОРЫЙ ВЫПОЛНЯЕТСЯ ПРИ
СОЗДАНИИ ОБЪЕКТА КЛАССА (ПЕРЕД
КОНСТРУКТОРОМ). ЭТО МОЖЕТ БЫТЬ
ПОЛЕЗНО, НАПРИМЕР, ДЛЯ
ИНИЦИАЛИЗАЦИИ ПЕРЕМЕННЫХ
ЭКЗЕМПЛЯРА, СОЗДАНИЯ НОВЫХ
ОБЪЕКТОВ ИЛИ УСТАНОВКИ
НАЧАЛЬНОГО СОСТОЯНИЯ.

СУЩЕСТВУЕТ ДВА ТИПА ИНИЦИАЛИЗАТОРОВ В JAVA (МОГУТ ИСПОЛЬЗОВАТЬСЯ ОДНОВРЕМЕННО):

- **Статический (static) инициализатор** - это блок кода, который выполняется при первой загрузке класса в память **JVM** (один раз). Он используется для инициализации статических переменных класса. Статический инициализатор можно определить с помощью ключевого слова **"static"** перед блоком кода.
- **Нестатический (instance) инициализатор** - это блок кода, который выполняется каждый раз при создании нового объекта класса. Он используется для инициализации переменных экземпляра класса. Нестатический инициализатор можно определить без ключевого слова **«static»**.

4. КАКИЕ ЕСТЬ ОСОБЕННОСТИ СТАТИЧЕСКИХ ПЕРЕМЕННЫХ И МЕТОДОВ?

- **Они принадлежат классу**, а не конкретному объекту класса. Это означает, что все объекты этого класса будут использовать одно и то же значение для статических переменных и методов.
- **Они могут быть вызваны без создания экземпляра класса**. Доступ к статическим элементам класса можно получить через имя класса.
- **Они могут быть перегружены** (например в дочернем классе), но не переопределены.
- **Статические методы могут быть одновременно абстрактными**, то есть в дочерних классах их необходимо реализовать.
- **Можно использовать статические методы внутри обычных методов**. Кроме того, обычные методы могут быть вызваны из статических методов, но только если они принадлежат к экземпляру класса (то есть необходимо определить объект).
- **В многопоточной среде их стоит использовать с осторожностью**, так как они имеют общее состояние и могут привести к неожиданному поведению программы и ошибкам выполнения.

5. КАКИЕ ОГРАНИЧЕНИЯ ДЕЙСТВУЮТ НА СТАТИЧЕСКИЕ ПЕРЕМЕННЫЕ И МЕТОДЫ?

- **Нельзя обращаться к нестатическим (instance) переменным и методам из статических методов или блоков кода.** Так как статический метод принадлежит классу, он может использовать только другие статические переменные и методы.
- Статические переменные и методы наследуются дочерними классами, но **не переопределяются**. Если дочерний класс определяет статическую переменную или метод с тем же именем, то он будет скрытой версией родительской.
- **Статические переменные и методы находятся в общем доступе** для всех экземпляров данного класса и для всех классов, которые имеют доступ к данному классу. Это может привести к конфликту имён.
- Статические переменные и методы могут **использоваться без создания объекта класса**, что означает, что эти переменные и методы всегда будут иметь общее состояние для всех объектов данного класса.
- Статические переменные и методы **доступны из любого места программы**, поэтому следует быть осторожным при работе со статическими переменными и методами и устанавливать правильные модификаторы доступа, чтобы обеспечить безопасность программы.

6. ЧТО ТАКОЕ АБСТРАКТНЫЙ (ABSTRACT) МЕТОД? — ЭТО МЕТОД, КОТОРЫЙ ОБЪЯВЛЕН, НО НЕ РЕАЛИЗОВАН В АБСТРАКТНОМ КЛАССЕ. ОН НЕ ИМЕЕТ ТЕЛА И ИСПОЛЬЗУЕТСЯ ДЛЯ ОПРЕДЕЛЕНИЯ СИГНАТУРЫ МЕТОДА И ТИПА ВОЗВРАЩАЕМОГО ЗНАЧЕНИЯ. ЭТО ОЗНАЧАЕТ, ЧТО ЛЮБОЙ КЛАСС, КОТОРЫЙ НАСЛЕДУЕТ АБСТРАКТНЫЙ КЛАСС, ДОЛЖЕН РЕАЛИЗОВАТЬ ВСЕ ЕГО АБСТРАКТНЫЕ МЕТОДЫ.

7. ЧТО ТАКОЕ АВТОУПАКОВКА И АВТОРАСПАКОВКА?

АВТОУПАКОВКА (AUTOBOXING) — ПРОЦЕСС АВТОМАТИЧЕСКОГО ПРЕОБРАЗОВАНИЯ ИЗ ПРИМИТИВНОГО ТИПА В СООТВЕТСТВУЮЩИЙ КЛАСС ОБЕРТКУ.

АВТОРАСПАКОВКА (UNBOXING) — ПРЕОБРАЗУЕТ КЛАСС ОБЕРТКУ В ПРИМИТИВ.

А ВОТ ЕСЛИ ОКАЖЕТСЯ ЗНАЧЕНИЕ ОБЕРТКИ **NULL**, ТО ПРИ РАСПАКОВКЕ БУДЕТ ВЫБРОШЕНО ИСКЛЮЧЕНИЕ **NULLPOINTEREXCEPTION**.

X.

**СТРОКИ (STRING) И ВСЕ ЧТО С НИМИ
СВЯЗАНО.**

1. ЧТО ТАКОЕ STRING? — ЭТО КЛАСС, КОТОРЫЙ ПРЕДСТАВЛЯЕТ ПОСЛЕДОВАТЕЛЬНОСТЬ СИМВОЛОВ. ОН ЯВЛЯЕТСЯ НЕИЗМЕНЯЕМЫМ (IMMUTABLE) ОБЪЕКТОМ, ЧТО ОЗНАЧАЕТ, ЧТО ЕГО ЗНАЧЕНИЕ НЕ МОЖЕТ БЫТЬ ИЗМЕНЕНО ПОСЛЕ СОЗДАНИЯ.

ХАРАКТЕРИСТИКИ STRING В JAVA:

- **Неизменяемость** - значения объекта String нельзя изменить после его создания. Это делает его безопасным для использования в многопоточном окружении и обеспечивает более простое управление памятью.
- **Unicode-кодировка** - в Java строки хранятся в формате Unicode, что позволяет использовать различные символы из разных языковых наборов.
- **Методы работы со строками** - класс String имеет много методов для работы со строками, таких как сравнение, поиск, замена, разделение, конкатенация строк и другие.
- **Пул строк** - Java использует пул строк (**string pool**), что позволяет экономить память и повышает производительность при работе со строками.
- **Использование в качестве ключей Map** - String часто используется в качестве ключей для Map, благодаря своей неизменяемости и возможности реализации методов **hashCode()** и **equals()**.
- **Создание объекта String** - объект String можно создать, используя литералы, конструкторы и методы. Он размещается в куче и может быть управляем сборщиком мусора.

2. ЧТО ТАКОЕ ПУЛ СТРОК? – ЭТО СПЕЦИАЛЬНАЯ ОБЛАСТЬ В ПАМЯТИ (HEAP), ГДЕ ХРАНЯТСЯ УНИКАЛЬНЫЕ СТРОКОВЫЕ ЛИТЕРАЛЫ, ЧТОБЫ ИЗБЕЖАТЬ ДУБЛИРОВАНИЯ И ЭКОНОМИТЬ ПАМЯТЬ.

ИСПОЛЬЗОВАНИЕ ПУЛА СТРОК ИМЕЕТ СЛЕДУЮЩИЕ ПРЕИМУЩЕСТВА:

- **Экономия памяти:** благодаря использованию пула строк, несколько строк с одинаковым значением будут использовать только один и тот же объект в памяти.
- **Быстродействие:** поиск в пуле строк занимает меньше времени, чем создание нового объекта, что может быть полезно в приложениях с большой нагрузкой.
- **Гарантированное поведение:** строковые литералы, которые объявлены в программе, всегда будут использовать пул строк и будут сравниваться между собой по значению, а не по ссылке.

ИНТЕРНИРОВАНИЕ — ЭТО ПРОЦЕСС ДОБАВЛЕНИЯ СТРОК В ПУЛ С ИСПОЛЬЗОВАНИЕМ **INTERN()**, ПРИ КОТОРОМ JVM ПРОВЕРЯЕТ НАЛИЧИЕ СТРОКИ В ПУЛЕ И ЕСЛИ ОНА ЕСТЬ, ВОЗВРАЩАЕТ ССЫЛКУ НА НЕЕ, ЕСЛИ НЕТ ТО СОХРАНЯЕТ ССЫЛКУ НА СТРОКУ В ПУЛЕ.

3. КАКАЯ РАЗНИЦА МЕЖДУ STRING, STRINGBUFFER И STRINGBUILDER?

- **String** - неизменяемый класс, предназначенный для работы со строками. Каждый раз, когда выполняется операция над объектом String, создается новый объект, что может привести к ухудшению производительности.
- **StringBuffer** - изменяемый класс для работы со строками. Он обеспечивает потокобезопасность и более медленный, чем StringBuilder.
- **StringBuilder** - также изменяемый класс для работы со строками. Он не обеспечивает потокобезопасность, но при этом более быстрый по сравнению с StringBuffer.

4. КАК СРАВНИТЬ ДВЕ СТРОКИ В JAVA И/ИЛИ ОТСОРТИРОВАТЬ ИХ?

- Метод **equals()** сравнивает содержимое двух строк и возвращает значение true, если они равны, и false - в противном случае.
- Метод **compareTo()** сравнивает две строки лексикографически и возвращает целое число, которое показывает, какая из строк больше или меньше. Если результат сравнения равен 0, это значит, что строки равны.
- Метод **Arrays.sort()** для сортировки массива строк. По умолчанию сортирует в алфавитном порядке, но можно использовать в аргументах свой компаратор (со своей логикой сортировки).

КАКИЕ УСЛОВИЯ ВЫДВИГАЮТСЯ ПО ПОВОДУ ПЕРЕОПРЕДЕЛЕНИЯ СДЕЛКИ ПРИ ПЕРЕОПРЕДЕЛЕНИИ EQUALS?

- **Рефлексивность:** `a.equals(a)` должно вернуть `true`. То есть объект всегда равен самому себе.
- **Симметричность:** если `a.equals(b)` вернуло `true`, то и `b.equals(a)` должно вернуть `true`.
- **Транзитивность:** если `a.equals(b)` и `b.equals(c)` вернули `true`, то и `a.equals(c)` должно вернуть `true`.
- **Консистентность:** повторные вызовы метода `equals()` для одного объекта должны возвращать одинаковый результат, при условии, что никакие поля, используемые при проверке на равенство, не были изменены.
- **Несравнимость с null:** `a.equals(null)` должно вернуть `false`.

СИММЕТРИЧНОСТЬ — ОТНОСИТСЯ К КОНТРАКТУ МЕТОДА `EQUALS()`, ТРЕБУЮЩЕМУ, ЧТОБЫ ДЛЯ ЛЮБЫХ НЕНУЛЕВЫХ ССЫЛОК `X` И `Y`, ЕСЛИ `X.EQUALS(Y)` ВОЗВРАЩАЕТ **TRUE**, ТО И `Y.EQUALS(X)` ДОЛЖНО ВОЗВРАЩАТЬ **TRUE**.

КРОМЕ ТОГО, ПЕРЕОПРЕДЕЛЯЯ МЕТОД EQUALS(),
НУЖНО УЧИТЫВАТЬ ТИП ПЕРЕДАВАЕМОГО АРГУМЕНТА И
ИСПОЛЬЗОВАТЬ ОПЕРАТОР **INSTANCEOF** ДЛЯ
ПРОВЕРКИ. ЕСЛИ ТИП АРГУМЕНТА ОТЛИЧАЕТСЯ ОТ ТИПА
ТЕКУЩЕГО ОБЪЕКТА, МЕТОД ДОЛЖЕН ВЕРНУТЬ FALSE.
ЕСЛИ ЖЕ ТИПЫ СОВПАДАЮТ, НЕОБХОДИМО
ВЫПОЛНИТЬ СРАВНЕНИЕ ВСЕХ ПОЛЕЙ, КОТОРЫЕ
ОПРЕДЕЛЯЮТ РАВЕНСТВО ОБЪЕКТОВ.

5. ДЛЯ ЧЕГО ИСПОЛЬЗУЮТ EQUALS AND HASHCODE В JAVA?

- Метод **equals()** используется для проверки равенства двух объектов. Для классов, которые не переопределили этот метод, он проверяет, являются ли два объекта ссылками на один и тот же объект в памяти.
- Метод **hashCode()** используется при работе с хеш-таблицами и другими алгоритмами, основанными на хеш-функциях. Он должен генерировать уникальный целочисленный код для каждого объекта класса. Это помогает быстро находить объекты в коллекции, используя хеш-функцию для поиска.

КОНТРАКТ МЕЖДУ МЕТОДАМИ **EQUALS() И **HASHCODE()** ЗАКЛЮЧАЕТСЯ В ТОМ, ЧТО ЕСЛИ ДВА ОБЪЕКТА РАВНЫ СОГЛАСНО МЕТОДУ **EQUALS()**, ТО ОНИ ДОЛЖНЫ ИМЕТЬ ОДИНАКОВЫЙ **HASHCODE()**.**

ОБРАТНОЕ ПРАВИЛО НЕ ВСЕГДА ВЕРНО: ДВА ОБЪЕКТА С ОДИНАКОВЫМ **HASHCODE() МОГУТ БЫТЬ НЕ РАВНЫМИ СОГЛАСНО МЕТОДУ **EQUALS()**.**

ПРИ ПЕРЕОПРЕДЕЛЕНИИ МЕТОДОВ EQUALS() И HASHCODE() СЛЕДУЕТ ПРИДЕРЖИВАТЬСЯ СЛЕДУЮЩИХ ПРАВИЛ:

- Если два объекта равны согласно методу **equals()**, то они должны иметь одинаковый **hashCode()**.
- Для двух любых объектов класса, для которых **equals()** возвращает **false**, не требуется, чтобы их **hashCode()** были разными, но это может увеличить эффективность работы с хеш-таблицами.

6. В ЧЕМ РАЗНИЦА МЕЖДУ == И EQUALS()?

- Оператор == (в основном работает с примитивами) сравнивает ссылки на объекты, то есть проверяет, указывают ли две переменные на один и тот же объект в памяти.
- Метод **equals()** сравнивает содержимое объектов, а не ссылки на них (в случае переопределения). Реализация метода **equals()** может быть переопределена для классов, чтобы определить, как должно быть выполнено сравнение содержимого.

XI.

COLLECTION FRAMEWORK.



1. КАКОВА ИЕРАРХИЯ JAVA COLLECTION FRAMEWORK?

- **Collection** - базовый интерфейс, предоставляющий методы для работы с группами объектов.
- **List** - интерфейс, представляющий упорядоченную коллекцию элементов, которые могут дублироваться.
- **Set** - интерфейс, представляющий неупорядоченную коллекцию уникальных элементов.
- **Queue** - интерфейс, представляющий коллекцию элементов, расположенных по порядку.
- **Deque** - интерфейс, представляющий двустороннюю очередь, в которой элементы могут добавляться и удаляться как с конца, так и с начала.
- **Map** - интерфейс, представляющий ассоциативную коллекцию пар «ключ-значение».
- **SortedSet** - интерфейс, представляющий отсортированное множество уникальных элементов.
- **SortedMap** - интерфейс, представляющий отсортированную ассоциативную коллекцию пар «ключ-значение».

2. РАССКАЖИТЕ ОБ ИТЕРАТОРАХ И ИХ ПРИМЕНЕНИИ.

ITERATOR — ЭТО ИНТЕРФЕЙС, ПРЕДСТАВЛЯЮЩИЙ СОБОЙ ОБЪЕКТ ДЛЯ ПОСЛЕДОВАТЕЛЬНОГО ПЕРЕБОРА ЭЛЕМЕНТОВ КОЛЛЕКЦИИ БЕЗ РАСКРЫТИЯ ЕЁ ВНУТРЕННЕЙ СТРУКТУРЫ.

ОСНОВНЫЕ МЕТОДЫ:

- **hasNext()** - проверяет, есть ли еще элементы в коллекции.
- **next()** - возвращает следующий элемент в коллекции.
- **remove()** - удаляет текущий элемент из коллекции.

**3. КАКОВО ВНУТРЕННЕЕ СТРОЕНИЕ
ARRAYLIST? — ПРИНЦИП РАБОТЫ
ЗАКЛЮЧАЕТСЯ В СОЗДАНИИ
МАССИВА ОПРЕДЕЛЕННОЙ ДЛИНЫ И
ПОСЛЕДУЮЩЕМ ЕГО ЗАПОЛНЕНИИ
ЭЛЕМЕНТАМИ. ЕСЛИ МАССИВ
СТАНОВИТСЯ НЕДОСТАТОЧНО
БОЛЬШИМ ДЛЯ ХРАНЕНИЯ НОВЫХ
ЭЛЕМЕНТОВ, ТО СОЗДАЕТСЯ НОВЫЙ
МАССИВ БОЛЬШЕГО РАЗМЕРА И ВСЕ
ЭЛЕМЕНТЫ КОПИРУЮТСЯ В НЕГО.**

ПРИ ЭТОМ, КОГДА ПРОИСХОДИТ ДОБАВЛЕНИЕ ИЛИ УДАЛЕНИЕ ЭЛЕМЕНТОВ ИЗ СЕРЕДИНЫ СПИСКА, ВСЕ ЭЛЕМЕНТЫ ПОСЛЕ ИЗМЕНЯЕМОГО СДВИГАЮТСЯ ВПРАВО ИЛИ ВЛЕВО СООТВЕТСТВЕННО.

КЛАСС ARRAYLIST ИМЕЕТ СЛЕДУЮЩИЕ ПОЛЯ:

- **elementData** - это массив, который используется для хранения элементов.
- **size** - это количество элементов в списке.
- **DEFAULT_CAPACITY** - это начальная емкость списка по умолчанию (10).
- **EMPTY_ELEMENTDATA** - это пустой массив, который используется при создании списка без указания начальной емкости.
- **MAX_ARRAY_SIZE** - это максимальный размер массива, который может быть создан в **Java ($2^{31} - 1$)**.

4. КАКОВО ВНУТРЕННЕЕ СТРОЕНИЕ LINKEDLIST? — ЭТО КЛАСС, КОТОРЫЙ ПРЕДСТАВЛЯЕТ СВЯЗАННЫЙ СПИСОК ЭЛЕМЕНТОВ. ВНУТРЕННЕ LINKEDLIST РЕАЛИЗОВАН КАК ДВУСВЯЗНЫЙ СПИСОК УЗЛОВ, КАЖДЫЙ ИЗ КОТОРЫХ СОДЕРЖИТ ССЫЛКИ НА СЛЕДУЮЩИЙ И ПРЕДЫДУЩИЙ УЗЛЫ В СПИСКЕ, А ТАКЖЕ ДАННЫЕ, ХРАНЯЩИЕСЯ В ЭТОМ УЗЛЕ.

КОГДА ЭЛЕМЕНТ ДОБАВЛЯЕТСЯ В LINKEDLIST, ОН СОЗДАЕТ НОВЫЙ УЗЕЛ, СОДЕРЖАЩИЙ ДАННЫЕ И ССЫЛКИ НА ПРЕДЫДУЩИЙ И СЛЕДУЮЩИЙ УЗЛЫ. ЭТОТ УЗЕЛ ЗАТЕМ ДОБАВЛЯЕТСЯ В СПИСОК ПУТЕМ ОБНОВЛЕНИЯ ССЫЛОК НА СОСЕДНИЕ УЗЛЫ В ЭТИХ УЗЛАХ.

5. КАКОВО ВНУТРЕННЕЕ УСТРОЙСТВО HASHMAP? — ОНО ОСНОВАНО НА ХЭШ-ТАБЛИЦАХ.

ХЭШ-ТАБЛИЦА — ЭТО СТРУКТУРА ДАННЫХ,
ХРАНЯЩАЯ ПАРЫ «КЛЮЧ-ЗНАЧЕНИЕ», КОТОРАЯ
ПОЗВОЛЯЕТ БЫСТРО И ЭФФЕКТИВНО ХРАНИТЬ
ПАРЫ КЛЮЧ-ЗНАЧЕНИЕ И ОБЕСПЕЧИВАЕТ ДОСТУП
К ЭТИМ ЗНАЧЕНИЯМ ЗА КОНСТАНТНОЕ ВРЕМЯ В
СРЕДНЕМ СЛУЧАЕ.

КАК РАБОТАЕТ HASHMAP:

- ▶ **Проверка инициализации массива:** При создании Map этот массив пуст. Как только вы вызываете первый `put()`, Java инициализирует массив дефолтным размером 16 ячеек.
- ▶ **Расчет хэш-кода ключа:** Java берет исходный 32-битный хэш-код объекта-ключа с помощью метода `key.hashCode()`. Если `key == null`, хэш-код всегда принудительно равен 0 (такие элементы всегда отправляются в нулевую ячейку массива).
- ▶ **Сглаживание хэша (Защита от коллизий):** Чтобы распределение по ячейкам было максимально случайным и равномерным, применяется функция сглаживания. Старшие 16 бит хэш-кода смешиваются с младшими с помощью операции:
`int hash = (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);`
- ▶ **Вычисление финального индекса ячейки:** Размер массива HashMap (n) всегда кратен степени двойки (16, 32, 64...). Чтобы мгновенно перевести сглаженный хэш в индекс массива, используется побитовая операция & (И) вместо медленного деления по модулю: `int index = (n - 1) & hash;`
- ▶ Если два ключа имеют одинаковый хэш-код (коллизия), они могут быть сохранены в одной ячейке массива, но будут храниться в односвязном списке в этой ячейке.
- ▶ Когда происходит запрос на получение значения по ключу, сначала вычисляется хэш-код ключа, затем определяется индекс массива, где может быть найдено значение. Если в этой ячейке есть список, пробегаем по списку, чтобы найти нужное значение.

**ВАЖНО ОТМЕТИТЬ, ЧТО ПРИ ИСПОЛЬЗОВАНИИ
HASHMAP НЕОБХОДИМО ПРАВИЛЬНО ПЕРЕОПРЕДЕЛИТЬ
МЕТОДЫ `EQUALS()` И `HASHCODE()` КЛАССА КЛЮЧА,
ЧТОБЫ ОБЕСПЕЧИТЬ ПРАВИЛЬНОЕ
ФУНКЦИОНИРОВАНИЕ ХЭШ-ТАБЛИЦЫ. КРОМЕ ТОГО,
КОГДА КОЛИЧЕСТВО ЭЛЕМЕНТОВ В HASHMAP
ДОСТИГАЕТ ОПРЕДЕЛЕННОГО ПОРОГА, РАЗМЕР
МАССИВА УВЕЛИЧИВАЕТСЯ АВТОМАТИЧЕСКИ ДЛЯ
ПОДДЕРЖАНИЯ ЭФФЕКТИВНОСТИ ХРАНЕНИЯ И
ДОСТУПА К ДАННЫМ.**

**6. КАКОГО ВНУТРЕННЕЕ СТРОЕНИЕ
TREESET? — ОНО ОСНОВАН НА
СТРУКТУРЕ ДАННЫХ TREEMAP
(КРАСНО-ЧЕРНОЕ ДЕРЕВО ПОИСКА),
ЧТО ОБЕСПЕЧИВАЕТ ХРАНЕНИЕ
УНИКАЛЬНЫХ ЭЛЕМЕНТОВ В
ОТСОРТИРОВАННОМ ПОРЯДКЕ.**

**КРАСНО-ЧЕРНОЕ ДЕРЕВО: РЕАЛИЗУЕТ
САМОБАЛАНСИРУЮЩЕЕСЯ БИНАРНОЕ ДЕРЕВО,
ГАРАНТИРУЮЩЕЕ, ЧТО ВРЕМЯ ОПЕРАЦИЙ
ОСТАЕТСЯ СТАБИЛЬНЫМ ДАЖЕ ПРИ БОЛЬШОМ
КОЛИЧЕСТВЕ ДАННЫХ.**

7. ЧЕМ ОТЛИЧАЕТСЯ ARRAYLIST ОТ LINKEDLIST?

- **ArrayList (для получения объектов по индексу):** использует массив для хранения элементов. Когда вы добавляете новый элемент в ArrayList, он добавляется в конец массива, если есть свободное место, или создается новый массив большего размера и все существующие элементы копируются в него. Это позволяет быстро получать элементы по индексу, потому что индекс соответствует индексу массива. **Однако это может занимать дополнительное время при добавлении или удалении элементов из середины списка, потому что нужно перемещать все элементы за измененным элементом, чтобы освободить или занять место.**
- **LinkedList (для добавления и удаления):** хранит элементы в виде узлов, каждый из которых содержит ссылку на следующий узел в списке. Это означает, что при добавлении или удалении элементов нет необходимости перемещать другие элементы, только нужно обновить ссылки на узлы. **Однако доступ к элементам по индексу выполняется медленнее, потому что для этого нужно пройти всю цепочку узлов до нужного индекса.**

8. ЧЕМ ОТЛИЧАЕТСЯ ARRAYLIST ОТ HASHSET?

- **ArrayList (добавление по порядку, могут быть дубликаты):** является списком, который хранит элементы по индексам в порядке добавления. Он поддерживает операции добавления элементов, удаления элементов, получения элементов по индексу и т.д. **По умолчанию ArrayList может содержать дубликаты элементов, то есть одинаковые значения могут быть добавлены несколько раз.**
- **HashSet (хранится не по порядку, не может быть дубликатов):** же является множеством, которое хранит элементы в случайном порядке. Он также поддерживает операции добавления, удаления и получения элементов, но не имеет индексов. **Кроме того, в отличие от ArrayList, не может содержать повторяющиеся элементы, то есть каждый элемент в множестве должен быть уникальным.**

9. ЗАЧЕМ В JAVA ТАКОЕ РАЗНООБРАЗИЕ ИМПЛЕМЕНТАЦИИ ДИНАМИЧЕСКОГО МАССИВА?

- **ArrayList** и **Vector** - это реализации динамического массива, которые позволяют хранить объекты в упорядоченном списке. Разница между ними заключается в том, что **Vector** является потокобезопасной имплементацией списка, в то время как **ArrayList** не является потокобезопасным. Таким образом, если требуется обращаться к списку из нескольких потоков, то следует использовать **Vector**.
- **LinkedList** - это имплементация списка, который является двунаправленным, что позволяет эффективно добавлять и удалять элементы в середине списка. Однако, если требуется часто производить доступ к элементу по индексу, то **ArrayList** может быть более эффективным выбором.

**ТАКЖЕ ЕСТЬ МНОЖЕСТВО ДРУГИХ СТРУКТУР ДАННЫХ,
ТАКИЕ КАК HASHSET, TREESET, HASHMAP, TREEMAP И
Т.Д.**

10. ЗАЧЕМ В JAVA ТАКОЕ РАЗНООБРАЗИЕ ИМПЛЕМЕНТАЦИИ KEY-VALUE STORAGE?

Например, если нужно быстро добавлять и извлекать элементы без учета порядка, можно использовать **HashMap**. Если нужно сохранять элементы в порядке их добавления, можно использовать **LinkedHashMap**. Если нужно сохранять элементы в отсортированном порядке ключей, можно использовать **TreeMap**.

ТАКЖЕ, В JAVA СУЩЕСТВУЕТ СТАНДАРТНЫЙ
ИНТЕРФЕЙС **MAP**, КОТОРЫЙ ИСПОЛЬЗУЕТСЯ ДЛЯ
РЕАЛИЗАЦИИ KEY-VALUE ХРАНИЛИЩ. ЭТОТ ИНТЕРФЕЙС
ОПРЕДЕЛЯЕТ ОБЩИЕ МЕТОДЫ ДЛЯ РАБОТЫ СО ВСЕМИ
ИМПЛЕМЕНТАЦИЯМИ.

**11. ЧТО ТАКОЕ КОЛЛИЗИЯ В
HASHCODE? — КОГДА ДВА РАЗНЫХ
ЗНАЧЕНИЯ ИМЕЮТ ОДИНАКОВЫЙ
ХЭШ-КОД ПРИ ИСПОЛЬЗОВАНИИ
ФУНКЦИИ ХЭШИРОВАНИЯ. ЭТО
МОЖЕТ ПРИВЕСТИ К ТОМУ, ЧТО
РАЗНЫЕ ЭЛЕМЕНТЫ БУДУТ
СОХРАНЕНЫ В ОДНОЙ И ТОЙ ЖЕ
ЯЧЕЙКЕ ТАБЛИЦЫ ХЕШЕЙ, ЧТО
МОЖЕТ ВЫЗВАТЬ КОНФЛИКТЫ ПРИ
ПОИСКЕ ЭЛЕМЕНТОВ.**

СУЩЕСТВУЕТ НЕСКОЛЬКО СПОСОБОВ БОРЬБЫ С КОЛЛИЗИЯМИ.

- **Один из них - это использование метода цепочек.** В этом случае каждая ячейка таблицы хешей содержит связанный список всех элементов, которые получаются с помощью той же функции хэширования. Если возникает коллизия, новый элемент добавляется в этот связанный список.
- **Другой способ - это использование метода открытой адресации.** При использовании этого метода, если возникает коллизия, новый элемент добавляется в следующую доступную ячейку таблицы хешей. Этот процесс повторяется до тех пор, пока не будет найдена свободная ячейка, в которую можно поместить элемент.
- **Третий способ - это изменение функции хэширования таким образом, чтобы она как можно меньше порождала коллизии.** Например, можно использовать более сложную функцию хэширования или увеличить размер таблицы хешей.

12. КАК СОРТИРОВАТЬ КОЛЛЕКЦИЮ ЭЛЕМЕНТОВ? — МОЖНО ОТСОРТИРОВАТЬ КОЛЛЕКЦИЮ ЭЛЕМЕНТОВ ПУТЕМ РЕАЛИЗАЦИИ ИНТЕРФЕЙСА COMPARABLE В КЛАССЕ ЭЛЕМЕНТОВ КОЛЛЕКЦИИ ИЛИ ПУТЕМ ПЕРЕДАЧИ ОБЪЕКТА COMPARATOR В МЕТОД СОРТИРОВКИ КОЛЛЕКЦИИ.

COMPARABLE — ЭТО ИНТЕРФЕЙС, КОТОРЫЙ ПОЗВОЛЯЕТ ЗАДАТЬ ЕСТЕСТВЕННЫЙ ПОРЯДОК СОРТИРОВКИ. КЛАСС ЭЛЕМЕНТОВ ДОЛЖЕН РЕАЛИЗОВАТЬ МЕТОД **COMPARETO()**, КОТОРЫЙ ВОЗВРАЩАЕТ ОТРИЦАТЕЛЬНОЕ ЧИСЛО(МЕНЬШИЙ), НОЛЬ (РАВНЫЙ) ИЛИ ПОЛОЖИТЕЛЬНОЕ ЧИСЛО(БОЛЬШИЙ).

COMPARATOR — ЭТО ИНТЕРФЕЙС, КОТОРЫЙ ПОЗВОЛЯЕТ ОПРЕДЕЛИТЬ ПОРЯДОК СОРТИРОВКИ ДЛЯ КЛАССА ЭЛЕМЕНТОВ БЕЗ НЕОБХОДИМОСТИ РЕАЛИЗОВЫВАТЬ ИНТЕРФЕЙС **COMPARABLE** ИЛИ ИЗМЕНЯТЬ ИСХОДНЫЙ КЛАСС ЭЛЕМЕНТОВ. КЛАСС, КОТОРЫЙ ВЫ ХОТИТЕ ИСПОЛЬЗОВАТЬ ДЛЯ СРАВНЕНИЯ ЭЛЕМЕНТОВ, ДОЛЖЕН РЕАЛИЗОВАТЬ ИНТЕРФЕЙС **COMPARATOR** И ПЕРЕДАВАТЬСЯ В МЕТОД СОРТИРОВКИ КОЛЛЕКЦИИ.

XII.

STREAM API.



**1. ЧТО ТАКОЕ STREAM (ПОТОК)? - ЭТО
ОБЪЕКТ, КОТОРЫЙ ПРЕДСТАВЛЯЕТ
СОБОЙ ПОСЛЕДОВАТЕЛЬНОСТЬ
ЭЛЕМЕНТОВ ДАННЫХ И ПОЗВОЛЯЕТ
ВЫПОЛНЯТЬ ОПЕРАЦИИ НАД ЭТИМИ
ЭЛЕМЕНТАМИ. ПОТОКИ
ПРЕДОСТАВЛЯЮТ ДЕКЛАРАТИВНЫЙ
СПОСОБ ОБРАБОТКИ ДАННЫХ БЕЗ
ИСПОЛЬЗОВАНИЯ ЦИКЛОВ.**

КЛЮЧЕВЫЕ ОСОБЕННОСТИ STREAM API:

- **Декларативный стиль:** Код описывает что сделать, а не как (скрывает циклы for/if).
- **Ленивые вычисления (Lazy):** Промежуточные операции (**filter, map**) не выполняются, пока не будет вызван терминальный метод (**collect, forEach**).
- **Конвейер операций:** Стрим состоит из источника, промежуточных операций и одной терминальной операции.
- **Неизменяемость (Immutability):** Стримы не меняют исходную коллекцию, а возвращают новый результат.

ПРОМЕЖУТОЧНЫЕ ОПЕРАЦИИ ВЫПОЛНЯЮТСЯ НАД ЭЛЕМЕНТАМИ ДАННЫХ И ВОЗВРАЩАЮТ НОВЫЙ ПОТОК.

ПРИМЕРЫ ТАКИХ ОПЕРАЦИЙ:

- **filter(Predicate<T> predicate):** Фильтрует элементы, оставляя только те, что соответствуют заданному условию (true).
- **map(Function<T, R> mapper):** Преобразует каждый элемент, применяя к нему функцию.
- **flatMap(Function<T, Stream<R>> mapper):** Преобразует каждый элемент в поток, а затем разворачивает (сплюсчивает) полученные потоки в один.
- **distinct():** Удаляет дубликаты, оставляя только уникальные элементы (использует equals()).
- **sorted() / sorted(Comparator<T> comparator):** Сортирует элементы в естественном или заданном порядке.
- **limit(long maxSize):** Ограничивает поток, оставляя только первые N элементов.
- **skip(long n):** Пропускает первые n элементов, возвращая оставшуюся часть.
- **peek(Consumer<T> action):** Выполняет действие над каждым элементом, не меняя поток (удобно для логирования/отладки).

ТЕРМИНАЛЬНЫЕ ОПЕРАЦИИ ЗАВЕРШАЮТ ОБРАБОТКУ ПОТОКА ДАННЫХ И ВОЗВРАЩАЮТ РЕЗУЛЬТАТ.

ПРИМЕРЫ ТАКИХ ОПЕРАЦИЙ:

- **collect(Collector)**: Преобразует поток в коллекцию (List, Set, Map) или строку.
- **forEach(Consumer)**: Выполняет действие для каждого элемента (часто для побочных эффектов, например, вывода в консоль).
- **reduce(BinaryOperator)**: Агрегирует элементы в одно значение (сумма, поиск макс).
- **count()**: Возвращает количество элементов в потоке (long).
- **min(Comparator) / max(Comparator)**: Находит минимальный/максимальный элемент.
- **findFirst() / findAny()**: Возвращает Optional с первым или любым элементом.
- **allMatch / anyMatch / noneMatch (Predicate)**: Проверяют условие для элементов, возвращают boolean.
- **toArray()**: Собирает элементы в массив.
- **forEachOrdered(Consumer)**: Как forEach, но гарантирует порядок элементов в параллельных потоках.

2. ОСНОВНЫЕ ОТЛИЧИЯ MAP И FLATMAP В JAVA STREAM API ЗАКЛЮЧАЮТСЯ В СПОСОБЕ ПРЕОБРАЗОВАНИЯ ЭЛЕМЕНТОВ:

- **map** производит **одно-к-одному** (трансформирует элемент), а **flatMap** — **одно-ко-многим** (преобразует элемент в поток и «сплющивает» структуру).
- **map** сохраняет структуру вложенности, тогда как **flatMap** объединяет их в один плоский поток.

XIII.

EXCEPTION, ERROR, LOGGING И ВСЯКАЯ
ОПАСНАЯ ДРЕБЕДЕНЬ.

1. ЧТО ТАКОЕ EXCEPTION? — ЭТО ОБЪЕКТ, КОТОРЫЙ ПРЕДСТАВЛЯЕТ ОШИБКУ ИЛИ ИСКЛЮЧИТЕЛЬНУЮ СИТУАЦИЮ ВО ВРЕМЯ ВЫПОЛНЕНИЯ ПРОГРАММЫ.

**КОГДА ВОЗНИКАЕТ ИСКЛЮЧЕНИЕ, ОНО
"БРОСАЕТСЯ" (THROWS) ИЗ ТЕКУЩЕГО МЕТОДА, И
ПРОГРАММА ИЩЕТ ПОДХОДЯЩИЙ "ОБРАБОТЧИК"
(HANDLER), ЧТОБЫ ОБРАБОТАТЬ ЭТО
ИСКЛЮЧЕНИЕ. ЕСЛИ ОБРАБОТЧИК НЕ НАЙДЕН, ТО
ПРОГРАММА ЗАВЕРШАЕТ СВОЮ РАБОТУ.**

ИСКЛЮЧЕНИЯ ДЕЛЯТСЯ НА ДВЕ КАТЕГОРИИ:

- ▶ **Checked исключения (наследники класса `Exception`)** - это те, которые должны быть обработаны программистом. Когда метод выбрасывает checked исключение, программа не скомпилируется, если не указано, как обработать это исключение. Это обеспечивает более надежную обработку ошибок в приложении и гарантирует, что любые потенциальные проблемы будут устранены до запуска кода.
- ▶ **Unchecked исключения (наследники класса `RuntimeException`)** - это те, которые не обязательно должны быть обработаны программистом. Unchecked исключения могут быть вызваны программой, но их отлавливание не обязательно. Некоторые примеры unchecked исключений включают в себя **`NullPointerException`** или **`ArrayIndexOutOfBoundsException`**.

ЛЮБОЙ КЛАСС, КОТОРЫЙ НАСЛЕДУЕТСЯ ОТ КЛАССА `EXCEPTION`, МОЖЕТ БЫТЬ ИСПОЛЬЗОВАН В КАЧЕСТВЕ ТИПА ИСКЛЮЧЕНИЯ.

2. КАКОВА ИЕРАРХИЯ ИСКЛЮЧЕНИЙ?

В JAVA, ИЕРАРХИЯ ИСКЛЮЧЕНИЙ НАЧИНАЕТСЯ С КЛАССА **THROWABLE**.

THROWABLE ИМЕЕТ ДВА ПОДКЛАССА: **ERROR** И **EXCEPTION**.

EXCEPTION ДЕЛИТСЯ НА **CHECKED** И **UNCHECKED** (**RUNTIMEEXCEPTION**).

ERROR ПРЕДСТАВЛЯЕТ СОБОЙ ОШИБКИ, КОТОРЫЕ ПРОИСХОДЯТ ВО ВРЕМЯ ВЫПОЛНЕНИЯ ПРИЛОЖЕНИЯ, КОТОРЫЕ НЕ ДОЛЖНЫ БЫТЬ ОБРАБОТАНЫ ПРОГРАММИСТОМ.

3. В ЧЕМ РАЗНИЦА МЕЖДУ ERROR И EXCEPTION?

Exception обычно возникает из-за ошибок в коде программы или некоторых внешних условий, таких как некорректный ввод пользователя, проблемы с соединением или файловой системой. Исключения должны быть обработаны программным кодом при помощи блока **try-catch** или выброса исключения для более высокого уровня.

С другой стороны, **Error** обычно возникает в критических ситуациях, связанных с работой JVM. Это могут быть проблемы с памятью, отказ жесткого диска, невозможность загрузки класса и т.д. Стандартная рекомендация для программирования на Java - не пытаться обрабатывать ошибки (**Error**), так как они обычно не поддаются коррекции на уровне программного кода.

Класс **Error** и его подклассы не требуют перехвата и обработки, поскольку они обычно возникают в критических ситуациях, когда дальнейшее выполнение программы может быть проблематичным. Обычно лучшим решением будет прервать выполнение программы и сообщить об ошибке пользователю или администратору системы.

4. КАКИЕ ОСОБЕННОСТИ ИСПОЛЬЗОВАНИЯ ОПЕРАТОРА TRY... CATCH?

- ▶ **Блок try** содержит код, который может породить исключение.
- ▶ **Блок catch** содержит код, который будет выполняться при возникновении исключения. Мы можем указать тип исключения, которое мы хотим обработать, и обрабатывать их по отдельности.
- ▶ **Один блок try может иметь несколько блоков catch**, каждый из которых обрабатывает определенный тип исключения.
- ▶ Можно использовать **блок finally**, который содержит код, который нужно выполнить в любом случае после завершения блока try-catch. Например, можно закрыть файл или соединение с базой данных в блоке finally.
- ▶ **Блок try-with-resources** содержит код, который автоматически закрывает используемые ресурсы, реализующие интерфейс AutoCloseable.
- ▶ Если исключение не было обработано в блоке try-catch, оно передается в более высокий уровень иерархии вызовов, где может быть обработано в другом блоке try-catch.

5. КАКИЕ ВОЗМОЖНЫЕ СПОСОБЫ ОБРАБОТКИ ИСКЛЮЧЕНИЙ ЕСТЬ?

- ▶ **Блок try-catch:** Для отлавливания возможного исключения при выполнении блока кода, и затем обработать это исключение в блоке catch.
- ▶ **Ключевое слово throws:** Если вы не хотите обрабатывать исключение в блоке try-catch, вы можете передать его наверх по стеку вызовов с помощью ключевого слова throws.
- ▶ **Блок finally:** Используется для выполнения кода независимо от того, было ли выброшено исключение или нет.
- ▶ **Конструкция try-with-resources:** Он позволяет автоматически закрыть ресурсы, которые были открыты в блоке try, после того как блок выполнится.
- ▶ **Ключевое слово throw:** Если вы хотите выбросить исключение в своем коде.
- ▶ **Обработка с помощью методов классов:** Некоторые классы, такие как Arrays или Collections, имеют методы для обработки исключений.
- ▶ **Создание пользовательских исключений:** Вы также можете создавать свои собственные исключения с помощью ключевого слова throw и наследуясь от класса Exception.

6. КАКОВЫ ПРАВИЛА ПРОВЕРКИ ИСКЛЮЧЕНИЙ ВО ВРЕМЯ НАСЛЕДОВАНИЯ?

- Подклассы могут выбрасывать только подклассы исключений, объявленные в суперклассе метода.
- Подклассы не могут выбрасывать новые проверяемые исключения, которые не упоминаются в суперклассе метода.
- Подклассы могут выбрасывать непроверяемые исключения любого типа, даже если этот тип не упоминается в сигнатуре метода суперкласса.
- Суперклассы не являются обязательными для выброса всех возможных исключений, указанных в сигнатуре метода.
- Если подкласс переопределяет метод, который не выбрасывает исключение, то подкласс может выбрасывать только непроверяемые исключения в этом методе.
- Если суперкласс выбрасывает несколько проверяемых исключений, то подкласс может выбрасывать любой из этих исключений или его подкласс.
- Когда метод переопределяется подклассом, он не может выбрасывать больше исключений, чем метод суперкласса, но может выбрасывать меньше или те же исключения, что и метод суперкласса.

7. КАКУЮ ЛОГИКУ МОЖНО ПРОПИСАТЬ В СВОИХ EXCEPTION?

- **Передача метаданных (Коды ошибок):** Вместо того чтобы парсить текстовое сообщение (`getMessage()`), добавьте поле с уникальным кодом ошибки или `enum`.
- **Контекстные данные:** Поля, которые помогут понять, что именно пошло не так. Например, ID заказа, который не нашелся, или список невалидных полей.
- **Сборка сообщения по шаблону:** Внутри конструктора можно инкапсулировать логику формирования красивого текста ошибки, чтобы не повторять её в коде бизнес-логики.
- **Интеграция с HTTP-статусами (для Spring):** Можно добавить логику определения HTTP-статуса (404, 400, 500) прямо в исключение или через аннотацию `@ResponseStatus`.
- **Логирование и подавление стектрейса:** Если исключение «бизнесовое» (например, «недостаточно средств»), оно не является программной ошибкой. В таких случаях можно переопределить метод `fillInStackTrace()`, чтобы он ничего не делал - это значительно ускорит создание объекта исключения.

XIV.

МНОГОПОТОЧНОСТЬ.



1. КАКИЕ ЕСТЬ СРЕДСТВА ДЛЯ РАБОТЫ С МНОГОПОТОЧНОСТЬЮ?

- ▶ **Класс Thread** - предоставляет самый базовый способ создания и управления потоками в Java.
- ▶ **Интерфейс Runnable** - позволяет определить задачу, которую может выполнить поток.
- ▶ **Класс ExecutorService**: Пул потоков. Предотвращает дорогое создание `new Thread()` на каждую задачу.
- ▶ **Классы Lock и Condition** из пакета `java.util.concurrent.locks` - предоставляют механизмы блокировки и синхронизации доступа к общим ресурсам.
- ▶ **Классы Semaphore и CyclicBarrier** из пакета `java.util.concurrent` - предоставляют дополнительные средства для управления поведением параллельного кода.
- ▶ **Классы AtomicBoolean, AtomicInteger и AtomicReference** из пакета `java.util.concurrent.atomic` - предоставляют безопасные атомарные операции над примитивными типами данных и объектами.
- ▶ **Классы CountdownLatch и Exchanger** из пакета `java.util.concurrent` - предоставляют дополнительные возможности для синхронизации потоков.
- ▶ **ConcurrentHashMap**: Вместо блокировки всей таблицы (как в старом Hashtable) блокирует отдельные бакеты/сегменты на запись. Чтение происходит без блокировок.

VOLATILE (ВИДИМОСТЬ) ИСПОЛЬЗУЕТСЯ ДЛЯ ПЕРЕМЕННЫХ. ОНО ГАРАНТИРУЕТ, ЧТО ИЗМЕНЕНИЯ, ВНЕСЕННЫЕ ОДНИМ ПОТОКОМ, БУДУТ НЕМЕДЛЕННО ВИДНЫ ДРУГИМ ПОТОКАМ.

- **Как работает:** Заставляет потоки читать и записывать значение напрямую в основную память, минуя кэш процессора.
- **Что дает:** Только видимость.
- **Чего не дает:** Не гарантирует атомарность. Операция **count++** с **volatile** переменной все равно может привести к потере данных, так как она состоит из трех шагов (чтение, инкремент, запись).
- **Когда использовать:** Для флагов состояния (например, **volatile boolean running = true**), которые один поток меняет, а другие проверяют.

SYNCHRONIZED (АТОМАРНОСТЬ + ВИДИМОСТЬ) МЕХАНИЗМ (МОНИТОРЫ), ОБЕСПЕЧИВАЕТ ВЗАИМНОЕ ИСКЛЮЧЕНИЕ: В ОДИН МОМЕНТ ВРЕМЕНИ ТОЛЬКО ОДИН ПОТОК МОЖЕТ ВЫПОЛНЯТЬ ЗАЩИЩЕННЫЙ КОД.

- **Как работает:** Поток захватывает «замок» (монитор) объекта. Пока он его держит, другие потоки блокируются.
- **Что дает:** атомарность (весь блок кода выполняется как одно целое), и видимость.
- **Минусы:**
 - ➡ Потоки могут долго ждать (блокировка «тяжелая»).
 - ➡ Нельзя проверить, свободен ли замок, не засыпая (**non-blocking**).
 - ➡ Нельзя прервать поток, который ждет входа в **synchronized**.

LOCKS (ГИБКОЕ УПРАВЛЕНИЕ) ИНТЕРФЕЙС **(ОСНОВНАЯ РЕАЛИЗАЦИЯ — REENTRANTLOCK)** **ПРЕДОСТАВЛЯЕТ БОЛЕЕ ПРОДВИНУТЫЙ КОНТРОЛЬ,** **ЧЕМ SYNCHRONIZED.**

- **tryLock():** Позволяет попытаться захватить блокировку. Если она занята, поток не засыпает, а может заняться чем-то другим.
- **lockInterruptibly():** Поток, ожидающий блокировку, можно прервать.
- **Честность (Fairness):** Можно включить очередь, чтобы блокировку получал тот поток, который ждет дольше всех.
- **ReadWriteLock:** Позволяет множеству потоков одновременно читать данные, но только одному — записывать. Это значительно ускоряет работу при преобладании чтений.

ВАЖНО: В ОТЛИЧИЕ ОТ SYNCHRONIZED, ПРОГРАММИСТ
ОБЯЗАН САМ ВЫЗВАТЬ **LOCK.UNLOCK()** В БЛОКЕ
FINALLY.

2. ЧТО ТАКОЕ ПРОЦЕСС? — ЭТО ПРОГРАММА ВО ВРЕМЯ ВЫПОЛНЕНИЯ. ОН ЯВЛЯЕТСЯ ЭКЗЕМПЛЯРОМ ПРОГРАММЫ, КОТОРАЯ ЗАПУСКАЕТСЯ НА КОМПЬЮТЕРЕ. КАЖДЫЙ ПРОЦЕСС ИМЕЕТ СВОЕ СОСТОЯНИЕ, КОТОРОЕ ВКЛЮЧАЕТ ДАННЫЕ, КОД И ДРУГИЕ СИСТЕМНЫЕ РЕСУРСЫ, ИСПОЛЬЗУЕМЫЕ ПРОГРАММОЙ.

3. ЧТО ТАКОЕ ПОТОК? – ЭТО ПОДПРОЦЕСС, КОТОРЫЙ РАБОТАЕТ ВНУТРИ ПРОЦЕССА. ПОТОКИ ВЫПОЛНЯЮТСЯ ПАРАЛЛЕЛЬНО, КАК БУДТО ОНИ ЯВЛЯЮТСЯ ОТДЕЛЬНЫМИ ПРОГРАММАМИ, НО ВСЕ ЕЩЕ МОГУТ ОБМЕНИВАТЬСЯ ДАННЫМИ И ДОСТУПОМ К РЕСУРСАМ ПРОЦЕССА. КАЖДЫЙ ПОТОК ИМЕЕТ СВОЙ СТЕК ВЫЗОВОВ И МОЖЕТ ВЫПОЛНЯТЬ НЕКОТОРУЮ ЧАСТЬ ГЛАВНОЙ ПРОГРАММЫ.

ОСНОВНОЕ ОТЛИЧИЕ МЕЖДУ ПРОЦЕССОМ И ПОТОКОМ:

ПРОЦЕСС — ЭТО НЕЗАВИСИМЫЙ ИСПОЛНЯЕМЫЙ
ОБЪЕКТ, КОТОРЫЙ ИМЕЕТ СВОЮ СОБСТВЕННУЮ
ОБЛАСТЬ ПАМЯТИ,

А **ПОТОК** — ЭТО ЛЕГКОВЕСНЫЙ ПОДПРОЦЕСС,
КОТОРЫЙ РАЗДЕЛЯЕТ РЕСУРСЫ (ПАМЯТЬ, ФАЙЛЫ И
Т.Д.) С ДРУГИМИ ПОТОКАМИ В РАМКАХ ОДНОГО
ПРОЦЕССА.

ОДИН И ТОТ ЖЕ ПРОЦЕСС МОЖЕТ ИМЕТЬ НЕСКОЛЬКО
ПОТОКОВ, КОТОРЫЕ МОГУТ ПАРАЛЛЕЛЬНО
ВЫПОЛНЯТЬСЯ В РАМКАХ ЭТОГО ПРОЦЕССА.

4. ЧТО ТАКОЕ СОСТОЯНИЕ ГОНКИ (RACE CONDITION)? — ЭТО БАГ, КОТОРЫЙ ВОЗНИКАЕТ В МНОГОПОТОЧНЫХ ПРОГРАММАХ, КОГДА НЕСКОЛЬКО ПОТОКОВ ОДНОВРЕМЕННО ПЫТАЮТСЯ ПОЛУЧИТЬ ДОСТУП К ОДНИМ И ТЕМ ЖЕ ОБЩИМ ДАННЫМ, И ПРИ ЭТОМ ХОТЯ БЫ ОДИН ИЗ НИХ ВЫПОЛНЯЕТ ЗАПИСЬ.

5. ЧТО ТАКОЕ ВЗАИМНАЯ БЛОКИРОВКА (DEADLOCK)? — ЭТО СИТУАЦИЯ В МНОГОПОТОЧНОЙ ПРОГРАММЕ, КОГДА ДВА ИЛИ БОЛЕЕ ПОТОКА БЕСКОНЕЧНО БЛОКИРУЮТ ДРУГ ДРУГА, ОЖИДАЯ ОСВОБОЖДЕНИЯ РЕСУРСОВ, КОТОРЫЕ ОНИ УДЕРЖИВАЮТ.

**6. ЧТО ТАКОЕ СИНХРОНИЗАЦИЯ
МЕЖДУ ПОТОКАМИ? — ЭТО
ПРОЦЕСС КООРДИНАЦИИ
ВЫПОЛНЕНИЯ КОДА В НЕСКОЛЬКИХ
ПОТОКАХ ДЛЯ ПРЕДОТВРАЩЕНИЯ
ГОНОК ДАННЫХ И ОБЕСПЕЧЕНИЯ
КОРРЕКТНОГО ДОСТУПА К ОБЩИМ
РЕСУРСАМ. СИНХРОНИЗАЦИЯ МЕЖДУ
ПОТОКАМИ МОЖЕТ БЫТЬ
ОСУЩЕСТВЛЕНА С ПОМОЩЬЮ
ОДНОВРЕМЕННОГО ДОСТУПА К
ОБЩЕМУ ОБЪЕКТУ МОНИТОРА.**

- **Методы wait(), notify() и notifyAll()** используются для координации выполнения кода во время ожидания некоторого условия или события, связанного с общим ресурсом. Они могут вызываться только из синхронизированного блока кода, который блокирует доступ к общему ресурсу, и используются для управления исполнением потоков.
- **Метод wait()** приостанавливает выполнение текущего потока и освобождает монитор, связанный с текущим объектом, на котором вызывается метод. Это позволяет другим потокам получить доступ к этому объекту и использовать его. Поток остается заблокированным до тех пор, пока другой поток не вызовет метод **notify()** или **notifyAll()** на том же мониторе.
- **Метод notify()** разблокирует один из потоков, ожидающих этот монитор. Если есть несколько потоков, ожидающих монитор, то не определено, какой из них будет разблокирован. Если нет ожидающих потоков, вызов метода **notify()** не приводит к никаким эффектам.
- **Метод notifyAll()** разблокирует все потоки, ожидающие этот монитор. Это дает возможность каждому потоку обновить свое состояние и перепроверить условия для продолжения работы.
- **Метод join()** используется для ожидания завершения выполнения другого потока. Когда поток вызывает метод **join()** на другом потоке, он блокируется до тех пор, пока поток, на котором был вызван метод **join()**, не завершится.

7. КАК ОСТАНОВИТЬ ПОТОК?

- **Вызов метода `interrupt()` на экземпляре класса `Thread`** - это устанавливает у потока флаг прерывания, который можно проверять в коде потока с помощью метода **`isInterrupted()`**. Поток может продолжать выполнение, если он не вызывал блокирующие операции или не проверял состояние флага прерывания.
- **Использование флагов `volatile` или `AtomicBoolean`** для управления циклом выполнения потока. Метод **`run()`** должен проверять значение флага и завершать свое выполнение, если он установлен.
- **Использование метода `stop()`** для принудительной остановки потока. Однако этот метод не рекомендуется к использованию, так как он может оставить системные ресурсы в непредсказуемом состоянии.
- **Использование метода `System.exit()`** для завершения всей программы, которая содержит потоки.
- **Использование метода `Thread.interrupt()`**, захваченного блокировкой, которая вызывает **`InterruptedException`**. Это позволяет обработать исключение и корректно завершить выполнение потока.

8. КАК МЕЖДУ ПОТОКАМИ ОБМЕНИВАТЬСЯ ДАННЫМИ?

- **Общие переменные** - каждый поток может иметь доступ к общим переменным, которые используются для передачи информации между потоками. Они должны быть атомарными или синхронизированными, чтобы избежать гонок данных.
- **Механизмы блокировки** - блокировки, такие как класс **Lock** или инструкция **synchronized**, могут использоваться для синхронизации доступа к общим ресурсам и предотвращения гонок данных.
- **Использование очередей** - очереди можно использовать для передачи сообщений между потоками. Каждый поток может читать из очереди или записывать в нее, чтобы передавать данные другому потоку.
- **Объекты типа Semaphore** - семафоры позволяют ограничивать количество потоков, которые могут получить доступ к общим ресурсам. С помощью методов **tryAcquire()** и **release()** можно управлять доступом к общим ресурсам.
- **Объекты типа CountdownLatch и CyclicBarrier** - это классы, позволяющие синхронизировать выполнение нескольких потоков.
- **Использование объектов типа BlockingQueue (ArrayBlockingQueue и LinkedBlockingQueue)** - это интерфейс, позволяющий использовать блокирующие операции для чтения или записи данных в очередь, что делает его безопасным для параллельной работы.

9. В ЧЕМ ОТЛИЧИЕ КЛАССА **THREAD** ОТ ИНТЕРФЕЙСА **RUNNABLE**?

ОСНОВНОЕ ОТЛИЧИЕ МЕЖДУ КЛАССОМ **THREAD** И ИНТЕРФЕЙСОМ **RUNNABLE** ЗАКЛЮЧАЕТСЯ В ТОМ, ЧТО КЛАСС **THREAD** ПРЕДОСТАВЛЯЕТ БОЛЬШУЮ ГИБКОСТЬ ПРИ УПРАВЛЕНИИ ПОТОКАМИ И ИХ ЖИЗНЕННЫМ ЦИКЛОМ, А ИНТЕРФЕЙС **RUNNABLE** ОБЕСПЕЧИВАЕТ БОЛЬШУЮ ГИБКОСТЬ В ОРГАНИЗАЦИИ КОДА И ЕГО СТРУКТУРИРОВАНИИ ПРИ РАБОТЕ С МНОЖЕСТВОМ ПОТОКОВ.

THREAD — ПРИ СОЗДАНИИ ЭКЗЕМПЛЯРА ЭТОГО КЛАССА, ОН НАСЛЕДУЕТ ВСЕ МЕТОДЫ И СВОЙСТВА ОБЪЕКТА THREAD, ТАКИЕ КАК **START()**, **RUN()** И ДРУГИЕ. СОЗДАНИЕ ПОТОКА ЧЕРЕЗ НАСЛЕДОВАНИЕ ОТ КЛАССА THREAD ПОЗВОЛЯЕТ ПРОЩЕ УПРАВЛЯТЬ ЖИЗНЕННЫМ ЦИКЛОМ ПОТОКА И ЕГО СОСТОЯНИЕМ.

RUNNABLE — ЭТО ИНТЕРФЕЙС, КОТОРЫЙ ОПРЕДЕЛЯЕТ ТОЛЬКО ОДИН МЕТОД **RUN()**. ДЛЯ ИСПОЛЬЗОВАНИЯ НЕОБХОДИМО СОЗДАТЬ НОВЫЙ ОБЪЕКТ, РЕАЛИЗУЮЩИЙ ДАННЫЙ ИНТЕРФЕЙС И ПЕРЕДАТЬ ЕГО В КАЧЕСТВЕ ПАРАМЕТРА КОНСТРУКТОРУ КЛАССА THREAD. ИСПОЛЬЗОВАНИЕ ИНТЕРФЕЙСА ПОЗВОЛЯЕТ БОЛЕЕ ГИБКО ОРГАНИЗОВАТЬ КОД ПРИ РАБОТЕ С МНОЖЕСТВОМ ПОТОКОВ, УПРОЩАЕТ ПРОЦЕСС НАСЛЕДОВАНИЯ И РАЗДЕЛЕНИЯ КОДА МЕЖДУ НЕСКОЛЬКИМИ ПОТОКАМИ.

10. РАЗНИЦА МЕЖДУ RUN() И START():

- **thread.start()** - создает новый поток в операционной системе и запускает в нем код из метода run() параллельно.
- **thread.run()** - не создает новый поток. Он просто выполняет код метода run() как обычный метод в том же самом (текущем) потоке, полностью блокируя его выполнение.

XV.

БАЗА ДАННЫХ.



1. ЧТО ТАКОЕ БАЗА ДАННЫХ (БД)? —
ЭТО ОРГАНИЗОВАННАЯ СТРУКТУРА,
ПРЕДНАЗНАЧЕННАЯ ДЛЯ ХРАНЕНИЯ,
ИЗМЕНЕНИЯ И ОБРАБОТКИ
ВЗАИМОСВЯЗАННОЙ ИНФОРМАЦИИ
ПРЕИМУЩЕСТВЕННО БОЛЬШИХ
ОБЪЕМОВ. ИНЫМИ СЛОВАМИ, БД —
ЭТО СТРУКТУРИРОВАННОЕ
ХРАНИЛИЩЕ ДАННЫХ. НАПРИМЕР,
ТЕЛЕФОННАЯ КНИГА.

ТИПЫ БД:

- **Реляционная база данных** — это набор данных с предопределенными связями между ними. Данные хранятся в виде набора таблиц, состоящих из столбцов и строк. В таблицах хранится информация об объектах, представленных в базе данных. В каждом столбце таблицы хранится определенный тип данных, в каждой ячейке — значение атрибута.
- **Нереляционные системы (NoSQL)** — системы, созданные для определенных моделей данных, обладающие гибкими схемами. Иначе говоря, это БД, которые хранят данные не в виде табличных схем, строк и столбцов, а в иных форматах.

**2. ЧТО ТАКОЕ СИСТЕМА УПРАВЛЕНИЯ
БАЗАМИ ДАННЫХ (СУБД)? — ЭТО
СОВОКУПНОСТЬ ПРОГРАММНОГО
ОБЕСПЕЧЕНИЯ, ПРИ ПОМОЩИ
КОТОРОГО ПОЛЬЗОВАТЕЛЬ МОЖЕТ
СОЗДАВАТЬ БАЗЫ ДАННЫХ (БД) И
ПРОВОДИТЬ НАД НИМИ РАЗЛИЧНЫЕ
ОПЕРАЦИИ: ДОПОЛНЯТЬ, ОБНОВЛЯТЬ,
УДАЛЯТЬ, ВЫБИРАТЬ, И Т. Д.**

3. НОРМАЛИЗАЦИЯ И ДЕНОРМАЛИЗАЦИЯ.

НОРМАЛИЗАЦИЯ — ЭТО ПРОЦЕСС ОРГАНИЗАЦИИ, СТРУКТУРИЗАЦИИ ДАННЫХ В БАЗЕ, КОТОРЫЙ ОБЕСПЕЧИВАЕТ БОЛЬШУЮ ГИБКОСТЬ БАЗЫ ДАННЫХ ЗА СЧЕТ ИСКЛЮЧЕНИЯ ИЗБЫТОЧНОСТИ И НЕСОГЛАСОВАННОСТИ ЗАВИСИМОСТЕЙ.

НОРМАЛЬНАЯ ФОРМА — СВОЙСТВО ТАБЛИЦЫ, РАССМАТРИВАЕМОЕ В КОНТЕКСТЕ НОРМАЛИЗАЦИИ, КОТОРОЕ ХАРАКТЕРИЗУЕТ ТАБЛИЦУ С ТОЧКИ ЗРЕНИЯ ПРОСТОТЫ И ПРАВИЛЬНОСТИ ПОСТРОЕНИЯ СТРУКТУРЫ. НОРМАЛЬНАЯ ФОРМА ОПРЕДЕЛЯЕТСЯ КАК СОВОКУПНОСТЬ ТРЕБОВАНИЙ, КОТОРЫМ ДОЛЖНА УДОВЛЕТВОРЯТЬ ТАБЛИЦА.

ВСЕГО СУЩЕСТВУЕТ ШЕСТЬ НОРМАЛЬНЫХ ФОРМ (НА ПРАКТИКЕ ИСПОЛЬЗУЮТ ПЕРВЫЕ ТРИ):

➡ ПЕРВАЯ НОРМАЛЬНАЯ ФОРМА:

- Все атрибуты простые (то есть атомарные и неделимые);
- Все данные скалярные (то есть положительные);
- Нет повторяющихся строк (для этого для каждой строки создается первичный ключ).

➡ ВТОРАЯ НОРМАЛЬНАЯ ФОРМА:

- Соблюдены условия первой нормальной формы;
- Каждый неключевой атрибут ссылается на первичный ключ.

➡ ТРЕТЬЯ НОРМАЛЬНАЯ ФОРМА:

- Соблюдены условия второй нормальной группы;
- Неключевые поля не зависят от других неключевых полей: они могут быть связаны лишь с первичным ключом.

ДЕНОРМАЛИЗАЦИЯ — НАМЕРЕННОЕ СНИЖЕНИЕ ИЛИ НАРУШЕНИЕ ФОРМ НОРМАЛИЗАЦИИ БАЗЫ ДАННЫХ, ОБЫЧНО — ЧТОБЫ УСКОРИТЬ ЧТЕНИЕ ИЗ БАЗЫ ЗА СЧЕТ ДОБАВЛЕНИЯ ИЗБЫТОЧНЫХ ДАННЫХ. В ОБЩЕМ, ЭТО ПРОЦЕСС, ОБРАТНЫЙ К НОРМАЛИЗАЦИИ.

ТАК ПРОИСХОДИТ ПОТОМУ, ЧТО ТЕОРИЯ НОРМАЛЬНЫХ ФОРМ НЕ ВСЕГДА ПРИМЕНИМА НА ПРАКТИКЕ.

4. ЧТО ТАКОЕ ИНДЕКС? —
ОТСОРТИРОВАННЫЙ НАБОР
ЗНАЧЕНИЙ, СВЯЗАННЫХ С ТАБЛИЦЕЙ
ИЛИ ПРЕДСТАВЛЕНИЕМ С
КОНКРЕТНОЙ КОЛОНКОЙ, КОТОРЫЙ
УСКОРЯЕТ ПОЛУЧЕНИЕ ДАННЫХ.

ВИДЫ ИНДЕКСОВ:

➔ **СОСТАВНОЙ** — ИНДЕКС ПОСТРОЕННЫЙ С ПОСЫЛАНИЕМ НА НЕСКОЛЬКО КОЛОНОК ОДНОВРЕМЕННО. ИНАЧЕ ГОВОРЯ, ЭТО КОМПЛЕКСНЫЙ ИНДЕКС, СОСТОЯЩИЙ ИЗ НЕСКОЛЬКИХ КОЛОНОК.

ТАКИЕ ИНДЕКСЫ ИСПОЛЬЗУЮТ, КОГДА В ОДНОМ ЗАПРОСЕ ФИГУРИРУЕТ БОЛЕЕ ОДНОЙ КОЛОНКИ.

➔ **ПОКРЫВАЮЩИЙ** — ЭТО ИНДЕКС, КОТОРОГО ВПОЛНЕ ДОСТАТОЧНО ДЛЯ ОТВЕТА НА ЗАПРОС БЕЗ ОБРАЩЕНИЯ К САМОЙ ТАБЛИЦЕ. ПО ЭТОМУ ИНДЕКСУ МОЖНО ДОСТАТЬ ВСЮ СТРОКУ ДАННЫХ, НО ПО ФАКТУ ЭТО ПРОСТО НЕ НУЖНО.

КЛАСТЕРИЗОВАННЫЙ:

- Обеспечивает физический порядок по выбранному полю;
- Если у таблицы есть кластеризованный индекс, она называется кластеризованной;
- Нужно не более одного индекса на таблицу;
- В MySQL кластеризованный индекс не задается явно пользователем, так как если вы не определяете PRIMARY KEY для своей таблицы, MySQL находит первый индекс **UNIQUE**, где все ключевые столбцы — **NOT NULL**, и InnoDB использует его в качестве кластеризованного индекса.

НЕКЛАСТЕРИЗОВАННЫЙ:

- В одной таблице возможно до 999 некластеризованных индексов;
- Содержит указатель на строки с реальными данными в таблице;
- Не обеспечивает физический порядок;
- Присутствуют отдельные таблицы с отсортированными данными, а именно — одна таблица для одного столбца, на котором индекс, поэтому при запросе данных, не входящих в состав данного поля, будет сначала выполняться запрос к полю в данной таблице, а только затем — дополнительный запрос к строке в изначальной таблице.

5. КЛЮЧИ И ИХ ПОВЕДЕНИЕ.

ПЕРВИЧНЫЙ КЛЮЧ (PRIMARY KEY)
— ПОЛЕ В ТАБЛИЦЕ ДЛЯ
ИДЕНТИФИКАЦИИ КАЖДОЙ СТРОКИ
В ТАБЛИЦЕ БАЗЫ ДАННЫХ.

ВНЕШНИЙ КЛЮЧ (FOREIGN KEY) —
ЭТО СВОЙСТВО, СОЗДАВАЕМОЕ ДЛЯ
ТОГО, ЧТОБЫ ОБЕСПЕЧИТЬ СВЯЗЬ
МЕЖДУ ТАБЛИЦАМИ.

ВОЗМОЖНЫЕ ВАРИАНТЫ ПОВЕДЕНИЯ:

- **CASCADE** — с этим свойством будут автоматически удалены или изменены строки из зависимой таблицы при удалении или изменении связанных строк в главной таблице;
- **SET NULL** — при данном свойстве, когда происходит удаление или обновление связанной строки из главной таблицы, будет устанавливаться значение NULL для столбца внешнего ключа;
- **NO ACTION** — отклоняет попытки удалить или изменить строки в главной таблице, если присутствуют связанные строки в зависимой таблице;
- **RESTRICT** — эквивалентно **NO ACTION**;
- **SET DEFAULT** — при данном свойстве, когда происходит удаление или обновление связанной строки из главной таблицы, будет устанавливаться значение по умолчанию (если оно есть) для столбца внешнего ключа.

6. ЧТО ТАКОЕ SQL-ИНЪЕКЦИЯ? — ОДИН ИЗ СПОСОБОВ ВЗЛОМА САЙТА, СУТЬ КОТОРОГО — ВНЕДРЕНИЕ В ДАННЫЕ ЧЕРЕЗ GET, POST ЗАПРОСЫ ИЛИ СООКИЕ НЕКОТОРОГО SQL КОДА. ЕСЛИ ВЕБ-САЙТ ВЫПОЛНЯЕТ ТАКИЕ ИНЪЕКЦИИ, МОЖНО ПОЛУЧИТЬ ДОСТУП К БД И ВЗЛОМАТЬ ПРИЛОЖЕНИЕ.

ВИДЫ ПРОЦЕДУРЫ В SQL:

- ➔ **ХРАНИМЫЕ ПРОЦЕДУРЫ В SQL** — СУЩНОСТЬ В БД, ПРЕДСТАВЛЯЮЩАЯ СОБОЙ НАБОР SQL-ИНСТРУКЦИЙ, КОТОРЫЙ КОМПИЛИРУЕТСЯ ОДИН РАЗ И ХРАНИТСЯ НА СЕРВЕРЕ. СЛОВОМ, ЭТО АНАЛОГ МЕТОДОВ В JAVA.
- ➔ **ХРАНИМАЯ ФУНКЦИЯ** — ЭТО РАЗНОВИДНОСТЬ ХРАНИМОЙ ПРОЦЕДУРЫ. ОТЛИЧИЕ ФУНКЦИИ СОСТОИТ В ТОМ, ЧТО ОНА ВСЕГДА ВОЗВРАЩАЕТ ТОЛЬКО ЕДИНИЧНОЕ ЗНАЧЕНИЕ, А ПРОЦЕДУРА — НАБОР ЗНАЧЕНИЙ.

➔ **ТРИГГЕР** — ЕЩЕ ОДНА РАЗНОВИДНОСТЬ ХРАНИМОЙ ПРОЦЕДУРЫ, КОТОРАЯ ВЫЗЫВАЕТСЯ НЕ НЕПОСРЕДСТВЕННО ПОЛЬЗОВАТЕЛЕМ, А ЕЕ АКТИВАЦИЯ ПРОИСХОДИТ ПРИ МОДИФИКАЦИИ ДАННЫХ. ТО ЕСТЬ ЭТА ПРОЦЕДУРА АКТИВИРУЕТСЯ ПРИ ВЫПОЛНЕНИИ НЕКОТОРЫХ УСЛОВИЙ, КАК, НАПРИМЕР, **INSERT** ИЛИ **DELETE**, ИЛИ **UPDATE** ДАННЫХ В ОПРЕДЕЛЕННОМ СТОЛБЦЕ ДАННОЙ ТАБЛИЦЫ.

МОМЕНТ ЗАПУСКА ТРИГГЕРА ОПРЕДЕЛЯЕТСЯ С ПОМОЩЬЮ КЛЮЧЕВЫХ СЛОВ **BEFORE** (ТРИГГЕР ЗАПУСКАЕТСЯ ДО СВЯЗАННОГО СОБЫТИЯ) ИЛИ **AFTER** (ПОСЛЕ СОБЫТИЯ).

7. ЧТО ТАКОЕ JDBC (JAVA DATABASE CONNECTIVITY)? – ЭТО СТАНДАРТНЫЙ ПРИКЛАДНОЙ ИНТЕРФЕЙС (API) В JAVA, КОТОРЫЙ ПОЗВОЛЯЕТ ПРОГРАММАМ ВЗАИМОДЕЙСТВОВАТЬ С РАЗЛИЧНЫМИ БАЗАМИ ДАННЫХ (СУБД) ЧЕРЕЗ УНИВЕРСАЛЬНЫЙ SQL-КОД.

8. ЧТО ТАКОЕ HIBERNATE И JPA?

HIBERNATE ORM — ЭТО ФРЕЙМВОРК ДЛЯ РАБОТЫ С РЕЛЯЦИОННЫМИ БАЗАМИ ДАННЫХ В JAVA. ОН ПРЕДОСТАВЛЯЕТ ОБЪЕКТНО-ОРИЕНТИРОВАННЫЙ ПОДХОД К РАБОТЕ С БАЗАМИ ДАННЫХ, ЧТО ПОЗВОЛЯЕТ РАЗРАБОТЧИКАМ ИЗБЕЖАТЬ НАПИСАНИЯ БОЛЬШОГО КОЛИЧЕСТВА SQL-ЗАПРОСОВ И УПРОЩАЕТ ВЗАИМОДЕЙСТВИЕ МЕЖДУ ПРИЛОЖЕНИЯМИ И БАЗОЙ ДАННЫХ.

JPA (JAVA PERSISTENCE API) — ЭТО СТАНДАРТ ДЛЯ РАБОТЫ С ОБЪЕКТНО-РЕЛЯЦИОННЫМ ОТОБРАЖЕНИЕМ (ORM) В JAVA. ОН ОПРЕДЕЛЯЕТ API ДЛЯ РАБОТЫ С БАЗАМИ ДАННЫХ ЧЕРЕЗ ORM. JPA НЕ ЯВЛЯЕТСЯ КОНКРЕТНОЙ РЕАЛИЗАЦИЕЙ ORM, А СКОРЕЕ СТАНДАРТИЗИРУЕТ РАБОТУ С НИМ.

В ЧЕМ РАЗНИЦА МЕЖДУ JPA И HIBERNATE?

РАЗНИЦА МЕЖДУ **JPA** И **HIBERNATE** ЗАКЛЮЧАЕТСЯ В ТОМ, ЧТО **JPA** ЯВЛЯЕТСЯ СТАНДАРТОМ, КОТОРЫЙ ИМЕЕТ НЕСКОЛЬКО РЕАЛИЗАЦИЙ, ВКЛЮЧАЯ **HIBERNATE**, **ECLIPSELINK** И **OPENJPA**.

HIBERNATE ORM — ОДНА ИЗ САМЫХ ПОПУЛЯРНЫХ РЕАЛИЗАЦИЙ **JPA** И ПРЕДОСТАВЛЯЕТ НАИБОЛЕЕ ШИРОКИЙ НАБОР ФУНКЦИОНАЛЬНЫХ ВОЗМОЖНОСТЕЙ И РАСШИРЕНИЙ. ОДНАКО, ИСПОЛЬЗОВАНИЕ **JPA** ПОЗВОЛЯЕТ СОЗДАВАТЬ БОЛЕЕ ПЕРЕНОСИМЫЙ КОД МЕЖДУ РАЗЛИЧНЫМИ ORM-ФРЕЙМВОРКАМИ, А ТАКЖЕ ПОВЫШАЕТ УРОВЕНЬ АБСТРАКЦИИ ВЗАИМОДЕЙСТВИЯ С БАЗОЙ ДАННЫХ.

**9. ЧТО ТАКОЕ
КАСКАДНОСТЬ(CASCADE)? — ЭТО
МЕХАНИЗМ В HIBERNATE,
ПОЗВОЛЯЮЩИЙ АВТОМАТИЧЕСКИ
РАСПРОСТРАНЯТЬ ОПЕРАЦИИ
СОХРАНЕНИЯ, ОБНОВЛЕНИЯ ИЛИ
УДАЛЕНИЯ СУЩНОСТИ НА
СВЯЗАННЫЕ С НЕЙ ОБЪЕКТЫ.**

hibernate поддерживает несколько типов каскадности:

- **CascadeType.ALL** - операция каскадного удаления, сохранения и обновления применяется ко всем связанным сущностям.
- **CascadeType.PERSIST** - каскадное сохранение применяется ко всем связанным сущностям.
- **CascadeType.MERGE** - каскадное обновление применяется ко всем связанным сущностям.
- **CascadeType.REMOVE** - каскадное удаление применяется ко всем связанным сущностям.
- **CascadeType.DETACH** - каскадное отсоединение применяется ко всем связанным сущностям.
- **CascadeType.REFRESH** - каскадное обновление применяется ко всем связанным сущностям.
- **CascadeType.NONE** - каскадность не применяется ни к одной связанной сущности.

10. ЧТО ТАКОЕ ENTITY MANAGER? — ЭТО ИНТЕРФЕЙС В JPA, КОТОРЫЙ ПРЕДОСТАВЛЯЕТ API ДЛЯ УПРАВЛЕНИЯ ЖИЗНЕННЫМ ЦИКЛОМ СУЩНОСТЕЙ. ENTITY MANAGER ОТВЕЧАЕТ ЗА УПРАВЛЕНИЕ СВЯЗЬЮ МЕЖДУ ОБЪЕКТАМИ JAVA И БАЗОЙ ДАННЫХ, ЧТО ПОЗВОЛЯЕТ РАЗРАБОТЧИКАМ ИСПОЛЬЗОВАТЬ ОБЪЕКТНО-ОРИЕНТИРОВАННЫЙ ПОДХОД ПРИ РАБОТЕ С БАЗОЙ ДАННЫХ.

ОСНОВНЫЕ ЗАДАЧИ ENTITY MANAGER ВКЛЮЧАЮТ:

- Создание, удаление и обновление сущностей в базе данных.
- Поиск и выборка сущностей из базы данных.
- Контроль жизненного цикла сущностей, таких как управление их состоянием (**managed, detached, transient**).
- Кэширование и оптимизация запросов к базе данных.
- Управление транзакциями.
- Работа с ленивой загрузкой (**lazy loading**) и Eager-загрузкой (**Eager loading**).

ENTITY MANAGER МОЖЕТ БЫТЬ ПОЛУЧЕН ЧЕРЕЗ **ENTITYMANAGERFACTORY**, КОТОРЫЙ СОЗДАЕТ И КОНФИГУРИРУЕТ СОЕДИНЕНИЕ С БАЗОЙ ДАННЫХ. ОБЪЕКТ **ENTITYMANAGER** ПРИВЯЗЫВАЕТСЯ К ОПРЕДЕЛЕННОЙ ТРАНЗАКЦИИ И УПРАВЛЯЕТ ДЕЛЕГИРОВАНИЕМ ИНСТРУКЦИЙ **SQL** В БАЗУ ДАННЫХ.

**11. ЧТО ТАКОЕ ТРАНЗАКЦИЯ
(TRANSACTION)? — ЭТО
ПОСЛЕДОВАТЕЛЬНОСТЬ ОПЕРАЦИЙ,
КОТОРЫЕ ВЫПОЛНЯЮТСЯ КАК
ЕДИНОЕ ЦЕЛОЕ И ЛИБО УСПЕШНО
ЗАВЕРШАЮТСЯ, ЛИБО
ОТКАТЫВАЮТСЯ К НАЧАЛЬНОМУ
СОСТОЯНИЮ В СЛУЧАЕ
ВОЗНИКНОВЕНИЯ ОШИБКИ.**

ГЛАВНЫЕ ХАРАКТЕРИСТИКИ ТРАНЗАКЦИЙ:

- **Атомарность (Atomicity)** - все операции транзакции должны быть выполнены или не выполнены вообще.
- **Согласованность (Consistency)** - транзакция должна приводить базу данных в согласованное состояние.
- **Изолированность (Isolation)** - каждая транзакция должна работать в изолированном режиме, т.е. изменения, внесенные одной транзакцией, не должны видны другим транзакциям до тех пор, пока первая транзакция не будет завершена.
- **Долговечность (Durability)** - после успешного завершения транзакции изменения должны сохраняться в базе данных.

**ACID-СВОЙСТВА — ТРАНЗАКЦИИ ДОЛЖНЫ БЫТЬ
АТОМАРНЫМИ, СОГЛАСОВАННЫМИ,
ИЗОЛИРОВАННЫМИ И ДОЛГОВЕЧНЫМИ.**

УРОВЕНЬ ИЗОЛЯЦИИ (ISOLATION LEVEL) —
ОПРЕДЕЛЯЕТ, НАСКОЛЬКО ТРАНЗАКЦИИ ДОЛЖНЫ
БЫТЬ ИЗОЛИРОВАНЫ ДРУГ ОТ ДРУГА.

В JAVA ЕСТЬ ЧЕТЫРЕ УРОВНЯ **ИЗОЛЯЦИИ:**

- **READ UNCOMMITTED (чтение незафиксированных данных)** - позволяет одной транзакции видеть изменения, которые еще не были зафиксированы другой транзакцией.
- **READ COMMITTED (чтение зафиксированных данных).**
- **REPEATABLE READ (повторяемое чтение).**
- **SERIALIZABLE (сериализуемость)** - обеспечивает полную изоляцию транзакций, при которой они ведут себя как будто выполняются последовательно, хотя фактически могут выполняться параллельно.

12. КАКАЯ РАЗНИЦА МЕЖДУ STATEMENT И PREPAREDSTATEMENT?

- **Statement** используется для создания статического SQL-запроса без параметров. Такой запрос выполняется каждый раз при вызове метода **execute()** объекта **Statement**.
- **PreparedStatement** (более производительный) же позволяет создавать динамический SQL-запрос с параметрами. Этот запрос компилируется только один раз, а затем может быть многократно выполнен с разными значениями параметров. Параметры указываются в виде плейсхолдеров **"?"** в **SQL-запросе**.

XVI.

**SQL SINTAXIS, SQL LANGUAGE И ВСЯКИЕ
SQL СЛОЖНОСТИ.**

1. ЧТО ТАКОЕ SQL (STRUCTURED QUERY LANGUAGE)? – ЭТО СТАНДАРТИЗИРОВАННЫЙ ЯЗЫК СТРУКТУРИРОВАННЫХ ЗАПРОСОВ, КОТОРЫЙ ИСПОЛЬЗУЕТСЯ ДЛЯ СОЗДАНИЯ, МОДИФИКАЦИИ И УПРАВЛЕНИЯ ДАННЫМИ В РЕЛЯЦИОННЫХ БАЗАХ ДАННЫХ.

2. КЛЮЧЕВЫЕ ГРУППЫ КОМАНД:

- **DDL (Data Definition Language):** Создание и изменение структуры базы данных (команды **CREATE**, **ALTER**, **DROP**).
- **DML (Data Manipulation Language):** Работа с непосредственными записями (команды **INSERT**, **UPDATE**, **DELETE**).
- **DQL (Data Query Language):** Используется исключительно для чтения и выборки данных (**SELECT** - главная и самая часто используемая команда в SQL для извлечения информации).
- **DCL (Data Control Language):** Управление правами доступа пользователей (**GRANT**, **REVOKE**).
- **TCL (Transaction Control Language):** Управление транзакциями для сохранения целостности данных (**COMMIT**, **ROLLBACK**).

3. ОСНОВНЫЕ ОПЕРАТОРЫ И ПРЕДЛОЖЕНИЯ ВНУТРИ SELECT:

- **FROM** - указывает главную таблицу, из которой берутся данные.
- **WHERE** - фильтрует строки по условию до того, как они будут объединены или сгруппированы.
- **JOIN** - связывает строки из двух или более таблиц на основе общего поля (ключа).
- **GROUP BY** - группирует строки с одинаковыми значениями (например, чтобы посчитать сумму или среднее).
- **HAVING** - фильтрует уже сгруппированные данные (работает как **WHERE**, но для групп).
- **ORDER BY** - сортирует итоговый результат (по возрастанию **ASC** или убыванию **DESC**).
- **LIMIT / OFFSET** - ограничивает количество выводимых строк (например, для страничного вывода).

4. СЕМЕЙСТВО ОПЕРАТОРОВ JOIN (СОЕДИНЕНИЯ):

- **INNER JOIN** - возвращает только те строки, у которых есть совпадения в обеих таблицах.
- **LEFT JOIN (или LEFT OUTER JOIN)** - возвращает все строки из левой таблицы и только совпавшие строки из правой. Если совпадений нет, подставляет **NULL**.
- **RIGHT JOIN** - зеркальный аналог **LEFT JOIN** (все строки из правой таблицы).
- **FULL JOIN** - возвращает строки из обеих таблиц, когда есть совпадение хотя бы в одной из них.
- **CROSS JOIN** - декартово произведение (каждая строка первой таблицы соединяется с каждой строкой второй).

5. АГРЕГАТНЫЕ ФУНКЦИИ:

ОНИ ИСПОЛЬЗУЮТСЯ ВМЕСТЕ С ПРЕДЛОЖЕНИЕМ **GROUP BY** ДЛЯ ПРОВЕДЕНИЯ РАСЧЕТОВ ПО ГРУППАМ СТРОК:

- **COUNT()** - подсчет количества строк.
- **SUM()** - вычисление суммы значений.
- **AVG()** - поиск среднего арифметического.
- **MIN()** / **MAX()** - поиск минимального и максимального значения.

6. ПОДЗАПРОСЫ (SUBQUERIES) И СТЕ:

ПОДЗАПРОСЫ: ЗАПРОС ВНУТРИ КРУГЛЫХ СКОБОК (НАПРИМЕР, В БЛОКЕ **WHERE** ИЛИ **FROM**).

СТЕ (COMMON TABLE EXPRESSIONS): КОНСТРУКЦИЯ **WITH**, КОТОРАЯ ПОЗВОЛЯЕТ СОЗДАТЬ ВРЕМЕННЫЙ ВИРТУАЛЬНЫЙ РЕЗУЛЬТАТ И ОБРАЩАТЬСЯ К НЕМУ КАК К ТАБЛИЦЕ. ЭТО ДЕЛАЕТ КОД ЧИТАЕМЫМ.

XVII.

SPRING FRAMEWORK.



7. ЧТО ТАКОЕ SPRING FRAMEWORK? —
ЭТО САМЫЙ ПОПУЛЯРНЫЙ
ФРЕЙМВОРК С ОТКРЫТЫМ
ИСХОДНЫМ КОДОМ ДЛЯ РАЗРАБОТКИ
КОРПОРАТИВНЫХ ПРИЛОЖЕНИЙ И
МИКРОСЕРВИСОВ НА ЯЗЫКЕ JAVA. ОН
УПРОЩАЕТ СОЗДАНИЕ СЛОЖНЫХ
СИСТЕМ, АВТОМАТИЗИРУЯ
ИНФРАСТРУКТУРНЫЕ ЗАДАЧИ И
ПОЗВОЛЯЯ РАЗРАБОТЧИКАМ
СОСРЕДОТОЧИТЬСЯ НА БИЗНЕС-
ЛОГИКЕ.

ОСНОВНЫЕ ХАРАКТЕРИСТИКИ И КОМПОНЕНТЫ SPRING:

- **Модульная структура:** Spring состоит из множества модулей (**Core, Web, Data, Security**), что позволяет подключать только необходимые компоненты.
- **IoC-контейнер (Inversion of Control):** Управляет жизненным циклом объектов (**beans**) и их зависимостями, освобождая разработчика от ручного создания объектов.
- **Spring Boot:** Популярное расширение, которое упрощает настройку и развертывание приложений, делая старт разработки максимально быстрым.
- **Spring Security:** Обеспечивает защиту приложений на уровне аутентификации и авторизации.
- **Spring Data:** Упрощает работу с базами данных (**SQL и NoSQL**).
- **Spring MVC:** Модуль для построения веб-приложений по модели «модель-представление-контроллер».

8. ЧТО ТАКОЕ ИОС-КОНТЕЙНЕР (INVERSION OF CONTROL)? — ЭТО ЦЕНТРАЛЬНЫЙ КОМПОНЕНТ ФРЕЙМВОРКА, КОТОРЫЙ АВТОМАТИЧЕСКИ СОЗДАЕТ, КОНФИГУРИРУЕТ, СОБИРАЕТ И УПРАВЛЯЕТ ЖИЗНЕННЫМ ЦИКЛОМ БИНОВ. ОН РЕАЛИЗУЕТ ПРИНЦИП ИНВЕРСИИ УПРАВЛЕНИЯ, ЗАБИРАЯ ОТВЕТСТВЕННОСТЬ ЗА СОЗДАНИЕ ЗАВИСИМОСТЕЙ У РАЗРАБОТЧИКА И ПЕРЕДАВАЯ ЕЕ ФРЕЙМВОРКУ, ЧТО ДЕЛАЕТ КОД БОЛЕЕ ГИБКИМ И ТЕСТИРУЕМЫМ.

ОСНОВНЫЕ ХАРАКТЕРИСТИКИ И ФУНКЦИИ:

- **Управление жизненным циклом:** Контейнер создает экземпляры объектов, внедряет зависимости и уничтожает их, когда они больше не нужны.
- **Dependency Injection (DI):** Контейнер автоматически внедряет необходимые зависимости в компоненты (через конструкторы, сеттеры или поля), используя информацию из конфигурации.
- **Конфигурация:** Объекты, которыми управляет контейнер, описываются с помощью аннотаций (например, **@Component**, **@Service**, **@Autowired**), Java-конфигурации или XML-файлов.
- **Типы контейнеров:** Spring предлагает два основных типа контейнеров: **BeanFactory** (базовый) и **ApplicationContext** (расширенный, наиболее часто используемый).

9. ЧТО ТАКОЕ ЖИЗНЕННЫЙ ЦИКЛ БИНА (BEAN LIFECYCLE)? — ЭТО ПОСЛЕДОВАТЕЛЬНОСТЬ ЭТАПОВ, ЧЕРЕЗ КОТОРЫЕ ПРОХОДИТ ОБЪЕКТ (БИН) ПОД УПРАВЛЕНИЕМ JOC-КОНТЕЙНЕРА: СОЗДАНИЕ, ВНЕДРЕНИЕ ЗАВИСИМОСТЕЙ, ИНИЦИАЛИЗАЦИЯ, ИСПОЛЬЗОВАНИЕ И УНИЧТОЖЕНИЕ.

ОСНОВНЫЕ ЭТАПЫ ЖИЗНЕННОГО ЦИКЛА БИНА:

- **Инстанцирование (Instantiation):** Контейнер создает экземпляры бина, используя конструктор.
- **Внедрение зависимостей (Populate Properties):** Spring автоматически внедряет зависимости (**@Autowired**).
- **Инициализация (Initialization):**
 - ➔ Вызываются методы **Aware** (если реализованы), например **BeanNameAware**, **BeanFactoryAware**.
 - ➔ Выполняются методы **postProcessBeforeInitialization** (**BeanPostProcessor**).
 - ➔ Вызываются методы инициализации: аннотация **@PostConstruct**, затем методы **InitializingBean** или кастомный **init-method**.
 - ➔ Выполняются методы **postProcessAfterInitialization** (**BeanPostProcessor**).
- **Использование (In Use):** Бин готов к работе и используется приложением.
- **Уничтожение (Destruction):** При закрытии контейнера (**context.close()**) вызываются методы уничтожения: **@PreDestroy**, **DisposableBean** или кастомный **destroy-method**.

10. ЧТО ТАКОЕ DEPENDENCY INJECTION (DI, ВНЕДРЕНИЕ ЗАВИСИМОСТЕЙ)?
— ЭТО ПАТТЕРН ПРОЕКТИРОВАНИЯ, ПРИ КОТОРОМ ОБЪЕКТ ПОЛУЧАЕТ СВОИ ЗАВИСИМОСТИ (ДРУГИЕ ОБЪЕКТЫ, С КОТОРЫМИ ОН РАБОТАЕТ) ИЗВНЕ (ЧЕРЕЗ КОНТЕЙНЕР), А НЕ СОЗДАЕТ ИХ САМОСТОЯТЕЛЬНО.

ОСНОВНЫЕ СПОСОБЫ ВНЕДРЕНИЯ:

- **Через конструктор (Constructor Injection):** Рекомендуемый способ. Зависимости передаются при создании объекта, что гарантирует их наличие и позволяет делать поля **final**.
- **Через сеттер (Setter Injection):** Зависимости устанавливаются через методы **set** после создания объекта. Удобно для необязательных зависимостей.
- **Через поле (Field Injection):** Аннотация **@Autowired** ставится непосредственно над полем. Самый простой, но менее предпочтительный способ (сложнее тестировать, скрывает зависимости).

SPRING ИСПОЛЬЗУЕТ APPLICATIONCONTEXT (IOC-КОНТЕЙНЕР) ДЛЯ УПРАВЛЕНИЯ ЖИЗНЕННЫМ ЦИКЛОМ ОБЪЕКТОВ, НАЗЫВАЕМЫХ БИНАМИ (BEANS).

ОСНОВНЫЕ ОБЛАСТИ ВИДИМОСТИ (SCOPES) БИНОВ:

➡ SINGLETON (ПО УМОЛЧАНИЮ)

- **Описание:** Контейнер Spring создает только один экземпляр бина на весь **IoC-контейнер**. Все запросы на получение этого бина вернут один и тот же кэшированный объект.
- **Применение:** Идеально для бинов без состояния (stateless), таких как сервисы, репозитории, контроллеры.
- **Особенность:** Создается при запуске контекста (если не указана ленивая инициализация).

➡ PROTOTYPE

- **Описание:** Новый экземпляр бина создается каждый раз, когда к нему обращаются (при инъекции или через **getBean()**).
- **Применение:** Используется для бинов, содержащих состояние, которое не должно разделяться между компонентами.
- **Особенность:** Spring не управляет полным жизненным циклом (не вызывает методы уничтожения).

**11. ЧТО ТАКОЕ SPRING AOP? — ЭТО
МОДУЛЬ ФРЕЙМВОРКА SPRING,
РЕАЛИЗУЮЩИЙ АСПЕКТНО-
ОРИЕНТИРОВАННОЕ
ПРОГРАММИРОВАНИЕ ДЛЯ
ОТДЕЛЕНИЯ СКВОЗНОЙ
ФУНКЦИОНАЛЬНОСТИ
(ЛОГИРОВАНИЕ, ТРАНЗАКЦИИ,
БЕЗОПАСНОСТЬ) ОТ БИЗНЕС-ЛОГИКИ.**

КЛЮЧЕВЫЕ КОНЦЕПЦИИ SPRING AOP:

- **Аспект (Aspect):** Модуль, объединяющий сквозную логику (например, класс с логированием).
- **Соединительная точка (Join Point):** Точка в выполнении программы, где может быть применен аспект, например, выполнение метода.
- **Срез (Pointcut):** Выражение, определяющее, к каким Join Point применять аспект.
- **Совет (Advice):** Действие, выполняемое аспектом до, после или вокруг метода (**Before, After, Around**).
- **Внедрение (Introduction):** Добавление новых методов или полей в существующие классы.
- **Связывание (Weaving):** Процесс связывания аспектов с целевыми объектами.

ОСНОВНЫЕ ПРЕИМУЩЕСТВА:

- **Уменьшение дублирования кода:** Общая логика выносится в аспекты.
- **Чистая бизнес-логика:** Основной код не засорен техническими деталями.
- **Гибкость:** Легко добавлять/убирать функциональность, не меняя код классов.

12. ЧТО ТАКОЕ SPRING DATA? — ЭТО СЕМЕЙСТВО ПРОЕКТОВ В РАМКАХ ЭКОСИСТЕМЫ SPRING, УПРОЩАЮЩЕЕ ДОСТУП К РЕЛЯЦИОННЫМ (SQL) И НЕРЕЛЯЦИОННЫМ (NOSQL) БАЗАМ ДАННЫХ. ПРЕДОСТАВЛЯЕТ АБСТРАКЦИЮ РЕПОЗИТОРИЕВ, АВТОМАТИЧЕСКУЮ ГЕНЕРАЦИЮ ЗАПРОСОВ ПО ИМЕНИ МЕТОДА И ЛЕГКУЮ ИНТЕГРАЦИЮ С РАЗЛИЧНЫМИ ХРАНИЛИЩАМИ.

ОСНОВНЫЕ ВОЗМОЖНОСТИ И ХАРАКТЕРИСТИКИ SPRING DATA:

- **Универсальная модель репозитория:** Доступ к данным осуществляется через интерфейсы (**Repository**, **CrudRepository**, **JpaRepository**), что избавляет от написания DAO-классов.
- **Автоматические запросы (Derived Queries):** Spring Data умеет создавать **SQL-запросы** на основе имен методов в интерфейсе, например, метод **findByLastName** автоматически найдет запись по фамилии.
- **Поддержка различных баз данных:** Существуют специализированные модули: **Spring Data JPA** (для реляционных БД), **Spring Data MongoDB**, **Spring Data Redis**, **Spring Data JDBC** и др..
- **Пагинация и сортировка:** Встроенные механизмы для удобного получения данных страницами (порциями) и их сортировки.
- **Минимальная конфигурация:** Особенно при использовании **Spring Boot**, подключение и настройка требуют минимум усилий.

**SPRING DATA ОБЕСПЕЧИВАЕТ РАБОТУ С ТРАНЗАКЦИЯМИ
ЧЕРЕЗ ДЕКЛАРАТИВНЫЙ ПОДХОД С ПОМОЩЬЮ
АННОТАЦИИ **@TRANSACTIONAL**, ГАРАНТИРУЯ
АТОМАРНОСТЬ ОПЕРАЦИЙ. ПРИМЕНЕНИЕ АННОТАЦИИ
НАД МЕТОДАМИ СЕРВИСОВ ИЛИ РЕПОЗИТОРИЕВ
ПОЗВОЛЯЕТ АВТОМАТИЧЕСКИ ОТКРЫВАТЬ
ТРАНЗАКЦИЮ, ФИКСИРОВАТЬ ЕЁ (**COMMIT**) ПРИ
УСПЕХЕ ИЛИ ОТКАТЫВАТЬ (**ROLLBACK**) ПРИ
ВОЗНИКНОВЕНИИ ИСКЛЮЧЕНИЙ, УПРОЩАЯ
УПРАВЛЕНИЕ БД.**

ОСНОВНЫЕ АСПЕКТЫ РАБОТЫ С ТРАНЗАКЦИЯМИ В SPRING DATA:

- **Декларативный подход (@Transactional):** Spring автоматически создает прокси для бинов, управляя открытием и закрытием соединений.
- **Уровень применения:** Аннотацию **@Transactional** можно ставить как над методами, так и над классами (сервисами), что обеспечивает выполнение всех методов в рамках одной транзакции.
- **Откат (Rollback):** По умолчанию транзакция откатывается при возникновении непроверяемых исключений (RuntimeException или Error). Для проверяемых исключений (checked exceptions) требуется явная настройка: **@Transactional(rollbackFor = Exception.class)**.
- **Менеджер транзакций:** Spring использует **PlatformTransactionManager** для управления подключением к базе данных (например, **JDBC, JPA**).
- **Программный подход:** В редких случаях (более тонкий контроль) используется **TransactionTemplate**.

13. ЧТО ТАКОЕ SPRING MVC (WEB)? —
ЭТО МОЩНЫЙ ВЕБ-ФРЕЙМВОРК В
СОСТАВЕ SPRING FRAMEWORK,
ОСНОВАННЫЙ НА ШАБЛОНЕ
ПРОЕКТИРОВАНИЯ MVC (MODEL-VIEW-
CONTROLLER). ОН ИСПОЛЬЗУЕТСЯ
ДЛЯ СОЗДАНИЯ ГИБКИХ,
СЛАБОСВЯЗАННЫХ JAVA-
ПРИЛОЖЕНИЙ, ВКЛЮЧАЯ
КЛАССИЧЕСКИЕ САЙТЫ, RESTFUL
СЕРВИСЫ, ПРЕДОСТАВЛЯЯ
ИНСТРУМЕНТЫ ДЛЯ ОБРАБОТКИ
HTTP-ЗАПРОСОВ И ГЕНЕРАЦИИ
ОТВЕТОВ.

ОСНОВНЫЕ ХАРАКТЕРИСТИКИ:

- **Архитектура MVC:** Разделяет приложение на модель (данные), вид (интерфейс) и контроллер (логика обработки), что упрощает разработку.
- **Центральный компонент:** **DispatcherServlet** работает как "Front Controller", перехватывая все запросы и распределяя их по обработчикам.
- **Аннотации:** Позволяет использовать **@Controller**, **@RequestMapping**, **@ResponseBody** для удобного создания контроллеров.
- **Гибкость:** Поддерживает различные типы представлений (**HTML, JSON, XML**), что удобно для создания **RESTful API**.
- **Интеграция:** Полностью интегрируется в экосистему **Spring Boot**.

14. ЧТО ТАКОЕ RESTFUL API В SPRING?

**— ЭТО АРХИТЕКТУРНЫЙ СТИЛЬ
СОЗДАНИЯ ВЕБ-СЕРВИСОВ,
ИСПОЛЬЗУЮЩИЙ ПРОТОКОЛ HTTP
ДЛЯ ОБМЕНА ДАННЫМИ МЕЖДУ
КЛИЕНТОМ И СЕРВЕРОМ В ФОРМАТЕ
JSON ИЛИ XML.**

**С ПОМОЩЬЮ SPRING BOOT И АННОТАЦИИ
@RESTCONTROLLER РАЗРАБОТЧИКИ ЛЕГКО
СОЗДАЮТ КОНЕЧНЫЕ ТОЧКИ ДЛЯ ОПЕРАЦИЙ CRUD
(СОЗДАНИЕ, ЧТЕНИЕ, ОБНОВЛЕНИЕ, УДАЛЕНИЕ),
ИСПОЛЬЗУЯ СТАНДАРТНЫЕ МЕТОДЫ HTTP (GET,
POST, PUT, DELETE).**

**15. ЧТО ТАКОЕ JDBCTEMPLATE? — ЭТО
ОСНОВНОЙ КЛАСС SPRING JDBC,
УПРОЩАЮЩИЙ РАБОТУ С БД ЗА СЧЕТ
УСТРАНЕНИЯ ШАБЛОННОГО КОДА
(ОТКРЫТИЕ/ЗАКРЫТИЕ
СОЕДИНЕНИЙ, ОБРАБОТКА
ИСКЛЮЧЕНИЙ).**

**ОН ИСПОЛЬЗУЕТСЯ ПУТЕМ ВНЕДРЕНИЯ
DATASOURCE В JDBCTEMPLATE, ВЫПОЛНЕНИЯ
SQL-ЗАПРОСОВ (QUERY, UPDATE) И МАППИНГА
РЕЗУЛЬТАТОВ С ПОМОЩЬЮ ROWMAPPER ИЛИ
RESULTSETEXTRACTOR.**

ОСНОВНЫЕ ЭТАПЫ ИСПОЛЬЗОВАНИЯ:

- **Настройка DataSource:** Убедитесь, что DataSource (источник данных) настроен как бин в **Spring IoC-контейнере**.
- **Создание JdbcTemplate:** Внедрите DataSource в JdbcTemplate, обычно в классе DAO/Repository.
- **Выполнение запросов:**
 - ➔ **update():** Для операций **INSERT, UPDATE, DELETE**.
 - ➔ **query():** Для выборки данных (**SELECT**), возвращает список объектов.
 - ➔ **queryForObject():** Для получения одного объекта.
- **МAPPING данных:** Используйте **RowMapper** для преобразования строк **ResultSet** в объекты Java.

16. ОСНОВНЫЕ ОТЛИЧИЯ

@COMPONENT ОТ @BEAN:

- **Применение:** @Component ставится над классом, а @Bean — над методом внутри класса @Configuration.
- **Автоматизация:** @Component позволяет Spring автоматически найти и зарегистрировать класс (Component Scan), а @Bean требует явного написания логики создания экземпляра.
- **Гибкость:** @Bean позволяет передавать параметры, вызывать конструкторы сторонних библиотек и применять сложную логику инициализации, что сложнее сделать с @Component.
- **Сторонний код:** Для создания бинов из сторонних библиотек (где нельзя добавить аннотацию к классу) используется исключительно @Bean.

17. ЧТО ТАКОЕ SPRING BOOT? — ЭТО ФРЕЙМВОРК, ПРЕДНАЗНАЧЕННЫЙ ДЛЯ БЫСТРОГО СОЗДАНИЯ ГОТОВЫХ К РАБОТЕ МИКРОСЕРВИСОВ И ПРИЛОЖЕНИЙ С МИНИМУМ НАСТРОЕК.

ОН АВТОМАТИЗИРУЕТ КОНФИГУРАЦИЮ КОМПОНЕНТОВ, ВКЛЮЧАЕТ ВСТРОЕННЫЕ ВЕБ-СЕРВЕРЫ (TOMCAT, JETTY) И УПРОЩАЕТ УПРАВЛЕНИЕ ЗАВИСИМОСТЯМИ ЧЕРЕЗ СТАРТОВЫЕ ПАКЕТЫ (STARTERS), СЛЕДУЯ ПРИНЦИПУ «КОНВЕНЦИЯ НАД КОНФИГУРАЦИЕЙ».

КЛЮЧЕВЫЕ ОСОБЕННОСТИ И ПРЕИМУЩЕСТВА:

- **Автоконфигурация:** Spring Boot автоматически настраивает приложение, анализируя подключенные библиотеки, что избавляет от написания большого количества конфигурационного кода.
- **Стартеры (Starters):** Набор зависимостей для упрощения настройки Maven/Gradle, например, **spring-boot-starter-web** для RESTful приложений.
- **Встроенные серверы:** Приложения собираются в исполняемые JAR-файлы со встроенным сервером (**Tomcat, Jetty**), поэтому не требуют отдельного веб-сервера.
- **Микросервисная архитектура:** Идеально подходит для создания независимых сервисов из-за легкости и скорости запуска.
- **Actuator:** Инструменты для мониторинга и управления приложением (метрики, состояние).

18. ЧТО ТАКОЕ КОНВЕНЦИЯ НАД КОНФИГУРАЦИЕЙ (CONVENTION OVER CONFIGURATION) В SPRING BOOT? — ЭТО ПРИНЦИП, ПРИ КОТОРОМ ФРЕЙМВОРК АВТОМАТИЧЕСКИ НАСТРАИВАЕТ ПРИЛОЖЕНИЕ, ОСНОВЫВАЯСЬ НА СТАНДАРТНЫХ СОГЛАШЕНИЯХ И НАЛИЧИИ БИБЛИОТЕК, МИНИМИЗИРУЯ РУЧНУЮ КОНФИГУРАЦИЮ.

ОСНОВНЫЕ АСПЕКТЫ ПРИНЦИПА В SPRING BOOT:

- **Автоконфигурация (Auto-Configuration):** Spring Boot анализирует **classpath** (добавленные зависимости) и автоматически конфигурирует бины. Например, если добавлен **spring-boot-starter-data-jpa**, фреймворк автоматически настроит **Hibernate** и пул соединений.
- **«Starters» (Стартеры):** Наборы зависимостей, которые автоматически подтягивают необходимые библиотеки, придерживаясь стандартных настроек.
- **Именованное и расположение:** Использование файлов **application.properties** или **application.yml** в `src/main/resources` для настройки параметров без явного кода.
- **Встроенный сервер:** Автоматическая настройка **Tomcat** или другого контейнера **сервлетов** «из коробки».

19. КАК РАБОТАЕТ

@SPRINGBOOTAPPLICATION:

- **@Configuration**: Помечает класс как источник определений бинов для контекста приложения (аналог XML-конфигурации).
- **@EnableAutoConfiguration**: Она ищет в **classpath** библиотеки (например, **spring-boot-starter-web**) и автоматически настраивает необходимые компоненты, например, подключает встроенный **Tomcat** или настраивает **DataSource**, если найден **Hibernate**.
- **@ComponentScan**: Автоматически сканирует текущий пакет и его подпакеты на наличие классов, помеченных аннотациями **@Component**, **@Service**, **@Repository**, **@Controller** и т.д., для регистрации их как бинов.

20. ОСНОВНЫЕ РАЗЛИЧИЯ @CONTROLLER И @RestController:

▸ Назначение:

➔ **@Controller**: Используется для традиционных MVC-контроллеров, которые возвращают представления (JSP, Thymeleaf).

➔ **@RestController**: Используется для **REST API**, где ответ — это данные (JSON, XML).

▸ Состав: **@RestController** — это аннотация-комбинация, эквивалентная **@Controller + @ResponseBody**.

▸ Обработка ответов:

➔ **@Controller**: Требуется явное указание **@ResponseBody**, если нужно вернуть данные, а не View.

➔ **@RestController**: Все методы по умолчанию пишут данные сразу в ответ.

XVIII.

HTTP/HTTPS, КОДЫ ОТВЕТОВ, ТЕЛА И
СУЩНОСТИ.

**1. ЧТО ТАКОЕ HTTP (HYPERTEXT
TRANSFER PROTOCOL)? — ЭТО
ПРИКЛАДНОЙ ПРОТОКОЛ ПЕРЕДАЧИ
ДАННЫХ, СОЗДАННЫЙ В 1991 ГОДУ
ТЁМОМ БЕРНЕРСОМ-ЛИ ДЛЯ ОБМЕНА
ТЕКСТОВЫМИ ДОКУМЕНТАМИ (HTML).
СЕГОДНЯ ОН РАБОТАЕТ ПО ПРИНЦИПУ
«КЛИЕНТ-СЕРВЕР»: БРАУЗЕР
ОТПРАВЛЯЕТ ЗАПРОС, А СЕРВЕР
ВОЗВРАЩАЕТ ОТВЕТ.**

**2. ЧТО ТАКОЕ HTTPS (HYPERTEXT
TRANSFER PROTOCOL SECURE)? —
ЭТО РАСШИРЕНИЕ ПРОТОКОЛА HTTP, В
КОТОРОЕ ДОБАВЛЕНА
КРИПТОГРАФИЧЕСКАЯ ЗАЩИТА
ДАННЫХ. РАНЬШЕ ИСПОЛЬЗОВАЛСЯ
ПРОТОКОЛ SSL, СЕГОДНЯ ЕГО
ПОЛНОСТЬЮ ЗАМЕНИЛ БОЛЕЕ
СОВРЕМЕННЫЙ И НАДЕЖНЫЙ TLS.**

3. КАК РАБОТАЕТ TLS-РУКОПОЖАТИЕ (HANDSHAKE):

- **Приветствие (Client/Server Hello):** браузер сообщает серверу, какие алгоритмы шифрования он поддерживает.
- **Передача сертификата:** сервер отправляет браузеру свой SSL/TLS-сертификат и свой открытый ключ.
- **Проверка:** браузер проверяет подлинность сертификата через встроенные в систему доверенные центры.
- **Генерация сессионного ключа:** браузер создает случайный секретный ключ, шифрует его полученным открытым ключом сервера и отправляет серверу.
- **Начало защищенной сессии:** сервер расшифровывает сообщение своим закрытым ключом. Теперь и у браузера, и у сервера есть общий секретный ключ. Все последующие данные шифруются только им.

SSL/TLS СЕРТИФИКАТЫ — ЭТО ЦИФРОВОЙ ПАСПОРТ САЙТА. БЕЗ НЕГО HTTPS РАБОТАТЬ НЕ БУДЕТ.

СЕРТИФИКАТЫ БЫВАЮТ ТРЕХ УРОВНЕЙ НАДЕЖНОСТИ:

- **DV (Domain Validation):** проверяется только владение доменом. Выпускается за пару минут (часто бесплатно через сервис Let's Encrypt). Подходит для 90% сайтов.
- **OV (Organization Validation):** проверяется существование реальной компании. Нужен бизнесу.
- **EV (Extended Validation):** глубокая юридическая проверка компании. Ранее подсвечивал название фирмы зеленой строкой в браузере. Используется банками и крупными корпорациями.

4. КАК УСТРОЕН HTTP-ЗАПРОС (REQUEST):

- **Стартовая строка:** указывает метод (что сделать), путь к странице и версию HTTP.
- **Заголовки:** служебная информация о браузере, языке и типе данных.
- **Пустая строка:** обязательный разделитель между заголовками и телом.
- **Тело запроса:** сами данные (например, логин и пароль при авторизации).

5. КАК УСТРОЕН HTTP-ОТВЕТ (RESPONSE):

- **Строка статуса:** показывает версию HTTP и код ответа (успех или ошибка).
- **Заголовки ответа:** метаданные сервера, дата и инструкции для браузера (например, сохранить куки).
- **Пустая строка:** обязательный разделитель.
- **Тело ответа:** контент, который отобразит браузер (HTML-код, картинка или JSON).

6. ОСНОВНЫЕ МЕТОДЫ (ДЛЯ РАБОТЫ С ДАННЫМИ):

- **GET** - Получить данные. Запрашивает информацию с сервера (например, открывает страницу сайта или скачивает картинку). Не имеет тела запроса и не должен изменять данные на сервере.
- **POST** - Создать новые данные. Отправляет данные на сервер для создания нового ресурса (например, регистрация пользователя, публикация поста или отправка формы). Данные передаются в теле запроса.
- **PUT** - Заменить данные полностью. Обновляет существующий ресурс, полностью заменяя его новыми данными. Если ресурса по указанному адресу нет, сервер может создать его с нуля.
- **PATCH** - Частично изменить данные. В отличие от PUT, этот метод обновляет только отдельные поля ресурса (например, меняет только ваш аватар в профиле, не затрагивая имя и пароль).
- **DELETE** - Удалить данные. Уничтожает указанный ресурс на сервере (например, удаляет товар из корзины или стирает аккаунт).

СЛУЖЕБНЫЕ МЕТОДЫ:

- **HEAD** - Запросить только заголовки. Аналогичен методу **GET**, но сервер возвращает ответ без тела(без самого HTML или картинки). Используется, чтобы быстро проверить, существует ли файл, какого он размера и когда обновлялся, без скачивания самого файла.
- **OPTIONS** - Узнать доступные методы. Запрашивает у сервера список методов, которые разрешено использовать для конкретного адреса. Часто автоматически отправляется браузерами перед основным запросом для проверки безопасности (**CORS**).
- **CONNECT** - Установить туннель. Превращает соединение в прозрачный TCP/IP-туннель. Обычно используется для установления защищенного **HTTPS-соединения** через прокси-серверы.
- **TRACE** - Проверить маршрут запроса. Возвращает клиенту обратно тот запрос, который он отправил. Используется для тестирования и диагностики, чтобы увидеть, вносят ли промежуточные серверы (прокси, файрволы) какие-либо изменения в запрос.

ВАЖНЫЕ СВОЙСТВА МЕТОДОВ:

- **Безопасные (Safe):** Не изменяют состояние данных на сервере. **GET, HEAD, OPTIONS.**
- **Идемпотентные:** Повторение одного и того же запроса много раз подряд дает один и тот же результат. **GET, HEAD, PUT, DELETE, OPTIONS.**
- **Могут иметь тело (Body):** Передают основной объем данных внутри себя. **POST, PUT, PATCH.**

МЕТОД POST НЕ ЯВЛЯЕТСЯ ИДЕМПОТЕНТНЫМ. ЕСЛИ ВЫ СЛУЧАЙНО НАЖМЕТЕ КНОПКУ «ОПЛАТИТЬ» ИЛИ «ОТПРАВИТЬ КОММЕНТАРИЙ» ПЯТЬ РАЗ ПОДРЯД, СЕРВЕР СОЗДАСТ ПЯТЬ РАЗНЫХ ТРАНЗАКЦИЙ ИЛИ КОММЕНТАРИЕВ.

7. КАКИЕ БЫВАЮТ КОДЫ ОТВЕТА:

1XX — ИНФОРМАЦИОННЫЕ (ПРОЦЕСС ИДЕТ).

- **100 Continue** - начало запроса принято, браузер может продолжать отправку данных.
- **101 Switching Protocols** - сервер согласен переключиться на другой протокол (например, на WebSockets).

2XX — УСПЕШНО (ВСЕ ХОРОШО).

- **200 OK** - самый частый код; запрос выполнен успешно, сервер вернул данные (страницу, картинку).
- **201 Created** - запрос выполнен успешно, и на сервере создан новый ресурс (например, зарегистрирован пользователь).
- **204 No Content** - запрос обработан успешно, но серверу нечего вернуть в теле ответа (часто используется при удалении данных).

3XX — ПЕРЕНАПРАВЛЕНИЕ / РЕДИРЕКТЫ (НУЖНО В ДРУГОЕ МЕСТО).

- **301 Moved Permanently** - страница навсегда переехала на новый адрес; браузеры запоминают его автоматически.
- **302 Found** - страница временно находится по другому адресу.
- **304 Not Modified** - страница не менялась с момента последнего посещения; браузер берет ее из своего кэша, экономя трафик.

4XX — ОШИБКА КЛИЕНТА (ВЫ ОШИБЛИСЬ).

- **400 Bad Gateway / Bad Request** - сервер не понял запрос, так как в нем допущена синтаксическая ошибка.
- **401 Unauthorized** - для доступа к странице требуется авторизация (логин и пароль).
- **403 Forbidden** - сервер понял запрос, но у вас категорически нет прав на просмотр этого ресурса.
- **404 Not Found** - страница по этому адресу не существует.
- **405 Method Not Allowed** - указанный метод (например, POST) запрещен для этого URL.
- **429 Too Many Requests** - пользователь отправил слишком много запросов за короткое время (защита от спама и DDOS).

5XX — ОШИБКА СЕРВЕРА (СЕРВЕР СЛОМАЛСЯ).

- **500 Internal Server Error** - любая непредвиденная ошибка на сервере (например, упал код сайта или сломалась база данных).
- **502 Bad Gateway** - один сервер (шлюз) получил некорректный ответ от другого сервера (например, Nginx не смог достучаться до Node.js или PHP).
- **503 Service Unavailable** - сервер временно не может обрабатывать запросы из-за перегрузки или технического обслуживания.
- **504 Gateway Timeout** - сервер не дождался ответа от вышестоящего сервера в отведенное время.

XIX.

ТЕСТИРОВАНИЕ.



**1. ЧТО ТАКОЕ МОДУЛЬНОЕ
ТЕСТИРОВАНИЕ (UNIT TESTING)? —
ЭТО ПРОЦЕСС ПРОВЕРКИ
КОРРЕКТНОСТИ РАБОТЫ
ОТДЕЛЬНЫХ, МИНИМАЛЬНО
ДЕЛИМЫХ ЧАСТЕЙ КОДА (МЕТОДОВ
ИЛИ КЛАССОВ) В ИЗОЛЯЦИИ ОТ
ОСТАЛЬНОЙ СИСТЕМЫ.**

ОСНОВНЫЕ АСПЕКТЫ МОДУЛЬНОГО ТЕСТИРОВАНИЯ В JAVA:

- **Цель:** Проверка правильности работы «юнита» (функционального модуля) и обнаружение ошибок на ранней стадии разработки.
- **Инструменты:** В Java стандартными фреймворками являются **JUnit** (особенно JUnit 5) и **TestNG**. Также часто используется библиотека **Mockito** для изоляции тестируемого кода (создания «заглушек»).
- **Характеристики тестов:** Модульные тесты должны быть быстрыми, независимыми друг от друга и детерминированными (стабильно проходить или падать).

ПРЕИМУЩЕСТВА:

- **Быстрое обнаружение багов:** Легче найти ошибку в методе, чем в целой системе.
- **Рефакторинг:** Позволяет менять код без страха сломать существующий функционал.
- **Документация:** Тесты показывают, как код должен работать.

2. ЧТО ТАКОЕ ИНТЕГРАЦИОННОЕ ТЕСТИРОВАНИЕ (INTEGRATION TESTING)? — ЭТО ЭТАП ПРОВЕРКИ, СЛЕДУЮЩИЙ ЗА ЮНИТ-ТЕСТАМИ, НА КОТОРОМ ОТДЕЛЬНЫЕ МОДУЛИ, СЕРВИСЫ ИЛИ КОМПОНЕНТЫ (НАПРИМЕР, СЕРВИС + БАЗА ДАННЫХ) ОБЪЕДИНЯЮТСЯ И ТЕСТИРУЮТСЯ ВМЕСТЕ.

КЛЮЧЕВЫЕ ОСОБЕННОСТИ И ИНСТРУМЕНТЫ:

- **Цель:** Выявление ошибок при обмене данными между компонентами (интерфейсами).
- **Что проверяется:** Взаимодействие с реальной или тестовой БД, внешними API, очередями сообщений.
- **Стек в Java:** Обычно используются **JUnit 5/TestNG, Spring Boot Test (@SpringBootTest)**, а также библиотека **Testcontainers** для запуска реальных баз данных в Docker-контейнерах.
- **Отличие от юнит-тестов:** Юнит-тесты проверяют изоляцию (один метод), а интеграционные — связку (взаимодействие).

3. В ЧЕМ РАЗНИЦА МЕЖДУ МОДУЛЬНЫМ И ИНТЕГРАЦИОННЫМ ТЕСТИРОВАНИЕМ?

МОДУЛЬНОЕ ТЕСТИРОВАНИЕ (UNIT TESTING)

ПРОВЕРЯЕТ РАБОТОСПОСОБНОСТЬ ОТДЕЛЬНЫХ, ИЗОЛИРОВАННЫХ КОМПОНЕНТОВ КОДА (ФУНКЦИЙ, МЕТОДОВ, КЛАССОВ), В ТО ВРЕМЯ КАК

ИНТЕГРАЦИОННОЕ ТЕСТИРОВАНИЕ (INTEGRATION TESTING)

ПРОВЕРЯЕТ, КАК ЭТИ КОМПОНЕНТЫ РАБОТАЮТ ВМЕСТЕ, ВЫЯВЛЯЯ ОШИБКИ ВЗАИМОДЕЙСТВИЯ.

МОДУЛЬНЫЕ ТЕСТЫ ВЫПОЛНЯЮТСЯ БЫСТРЕЕ И ЧАЩЕ,

А ИНТЕГРАЦИОННЫЕ — МЕДЛЕННЕЕ, ТАК КАК ТРЕБУЮТ НАСТРОЙКИ ОКРУЖЕНИЯ.

ОСНОВНЫЕ ОТЛИЧИЯ:

- **Цель:** Модульные тесты проверяют логику внутри одного модуля. Интеграционные тесты проверяют связи и передачу данных между модулями.
- **Изоляция:** Модульные тесты изолируют код от внешних зависимостей (баз данных, сети), используя заглушки (stubs). Интеграционные тесты работают с реальными или приближенными к реальным компонентами.
- **Сложность и скорость:** Модульные тесты обычно простые и быстрые. Интеграционные тесты более комплексные, требуют времени на настройку и выполнение.
- **Порядок:** Сначала проводятся модульные тесты, затем — интеграционные.
- **Кто пишет:** Модульные тесты — чаще всего разработчики, интеграционные — разработчики или QA-инженеры.

**4. ЧТО ТАКОЕ ПИРАМИДА
ТЕСТИРОВАНИЯ? — ЭТО СТРАТЕГИЯ
ОБЕСПЕЧЕНИЯ КАЧЕСТВА,
РЕКОМЕНДУЮЩАЯ ПИСАТЬ МНОГО
БЫСТРЫХ, НИЗКОУРОВНЕВЫХ ЮНИТ-
ТЕСТОВ (JUNIT, МОСКИТО) В ОСНОВЕ,
МЕНЬШЕ ИНТЕГРАЦИОННЫХ ТЕСТОВ
(SPRING BOOT TEST, TESTCONTAINERS)
ПОСЕРЕДИНЕ И МИНИМУМ СЛОЖНЫХ
UI/E2E-ТЕСТОВ (SELENIUM, SELENIDE)
НА ВЕРШИНЕ.**

УРОВНИ ПИРАМИДЫ ТЕСТИРОВАНИЯ.

➔ Unit Tests (Модульные тесты):

- **Суть:** Проверка минимальных частей кода (методов, классов) в изоляции.
- **Объем:** Самый большой (основание пирамиды).
- **Инструменты:** JUnit 5, Mockito, AssertJ.
- **Скорость:** Очень быстрые.

➔ Integration Tests (Интеграционные тесты):

- **Суть:** Проверка взаимодействия между компонентами (сервисы, БД, внешние API).
- **Объем:** Средний.
- **Инструменты:** Spring Boot Test, Testcontainers (для Docker-контейнеров), REST-assured.
- **Скорость:** Медленнее юнит-тестов.

➔ End-to-End Tests (UI/E2E Тесты):

- **Суть:** Тестирование сценариев пользователя от начала до конца (сквозное).
- **Объем:** Самый маленький (вершина).
- **Инструменты:** Selenium, Selenide.
- **Скорость:** Медленные, хрупкие.

1. ЧТО ТАКОЕ JUNIT? – ЭТО ВЕДУЩИЙ ОТКРЫТЫЙ ФРЕЙМВОРК ДЛЯ МОДУЛЬНОГО ТЕСТИРОВАНИЯ (UNIT TESTING) JAVA-ПРИЛОЖЕНИЙ, ОБЕСПЕЧИВАЮЩИЙ АВТОМАТИЗАЦИЮ ПРОВЕРКИ КОДА ЧЕРЕЗ АННОТАЦИИ И УТВЕРЖДЕНИЯ (ASSERTIONS).

КЛЮЧЕВЫЕ ОСОБЕННОСТИ JUNIT:

- **Модульность:** JUnit 5 состоит из платформы для запуска, **Jupiter** (API для новых тестов) и **Vintage** (для старых тестов JUnit 3/4).
- **Аннотации:** Используются для определения тестов (**@Test**), настроек (**@BeforeEach**, **@AfterEach**) и управления процессом (**@Disabled**, **@DisplayName**).
- **Утверждения (Assertions):** **Assertions.assertEquals(...)** и другие методы из `org.junit.jupiter.api.Assertions` проверяют ожидаемые результаты.
- **Параметризованные тесты:** **@ParameterizedTest** позволяет запускать один тест с разными наборами данных.
- **Расширяемость:** Модель расширений позволяет добавлять сторонние инструменты (например, **Mockito** для создания мок-объектов).

ОСНОВНЫЕ АННОТАЦИИ JUNIT 5:

- **@Test** - обозначает метод как тестовый.
- **@BeforeEach** / **@AfterEach** - выполняется перед/после каждого теста.
- **@BeforeAll** / **@AfterAll** - выполняется перед/после всех тестов в классе (должны быть static).
- **@DisplayName** - задает понятное имя для теста.
- **@Disabled** - временно отключает тест.

ОСНОВНЫЕ МЕТОДЫ ASSERT В JUNIT 5 (ASSERTIONS):

- **assertEquals(expected, actual):** Проверяет, что ожидаемое значение равно фактическому.
- **assertTrue(condition)/assertFalse(condition):** Проверяет, что условие истинно или ложно.
- **assertNull(object)/assertNotNull(object):** Проверяет, является ли объект null или нет.
- **assertSame(expected, actual)/assertNotSame:** Проверяет, ссылаются ли переменные на один и тот же объект.
- **assertThrows(ExpectedException.class, () -> { ... }):** Проверяет, что выполнение блока кода вызывает ожидаемое исключение.
- **assertAll(executable...):** Группирует несколько утверждений, выполняя их все, даже если одно из них провалилось.
- **assertTimeout(duration, executable):** Проверяет, что код выполняется в течение заданного времени.

5. ЧТО ТАКОЕ МОСК (ИМИТАЦИЯ, ЗАГЛУШКА)? — ЭТО СПЕЦИАЛЬНО СОЗДАННЫЙ ИСКУССТВЕННЫЙ ОБЪЕКТ ИЛИ ФУНКЦИЯ, ИМИТИРУЮЩАЯ ПОВЕДЕНИЕ РЕАЛЬНОГО КОМПОНЕНТА (БАЗЫ ДАННЫХ, API-ЗАПРОСА, СЕРВИСА) ПРИ ТЕСТИРОВАНИИ ПО.

ОСНОВНЫЕ АСПЕКТЫ МОК-ОБЪЕКТОВ:

- **Изоляция:** Моки позволяют тестировать конкретную часть программы, не запуская реальные внешние системы.
- **Управление:** Позволяют задать ожидаемые ответы (например, вернуть **"404 Ошибка"** вместо реальных данных).
- **Проверка взаимодействия:** В отличие от стабов, моки могут фиксировать, был ли вызван метод, сколько раз и с какими аргументами.

STUB ПРОСТО ВОЗВРАЩАЕТ ЗАРАНЕЕ ОПРЕДЕЛЕННЫЕ ДАННЫЕ. **МОСК** — БОЛЕЕ ПРОДВИНУТЫЙ ОБЪЕКТ, КОТОРЫЙ ПРОВЕРЯЕТ, КАК ИМЕННО ЕГО ИСПОЛЬЗОВАЛИ (КАКИЕ МЕТОДЫ ВЫЗЫВАЛИСЬ, С КАКИМИ АРГУМЕНТАМИ И СКОЛЬКО РАЗ).

ОСНОВНЫЕ АННОТАЦИИ MOSKITO:

- **@Mock**: Создает полный фиктивный объект (мок) класса или интерфейса. Методы такого объекта по умолчанию ничего не делают или возвращают **null/значения по умолчанию**.
- **@InjectMocks**: Создает экземпляр тестируемого класса и автоматически внедряет в него зависимые моки, помеченные аннотациями **@Mock** или **@Spy (через конструктор, сеттеры или поля)**.
- **@Spy**: Создает частичный мок («шпион») реального объекта. Вызовы методов по умолчанию выполняют реальную логику, но их можно переопределить (замокать).
- **@Captor**: Используется для создания экземпляра **ArgumentCaptor**, который позволяет захватить аргументы метода для последующей проверки в **verify**.

VERIFY() В MOSKITO — ЭТО МЕТОД ДЛЯ ПРОВЕРКИ
ВЗАИМОДЕЙСТВИЙ С МОК-ОБЪЕКТАМИ,
ПОДТВЕРЖДАЮЩИЙ, ЧТО КОНКРЕТНЫЙ МЕТОД БЫЛ
ВЫЗВАН ОПРЕДЕЛЕННОЕ КОЛИЧЕСТВО РАЗ С
ЗАДААННЫМИ АРГУМЕНТАМИ.

ОСНОВНЫЕ ВОЗМОЖНОСТИ И СИНТАКСИС VERIFY():

➔ **Базовая проверка:** Проверяет, был ли метод вызван хотя бы раз. `verify(mockObject).methodName();`.

➔ **Количество вызовов:**

- `verify(mockObject, times(2)).methodName();` - метод вызван ровно 2 раза.
- `verify(mockObject, never()).methodName();` - метод не вызывался.
- `verify(mockObject, atLeastOnce()).methodName();` - метод вызван хотя бы раз.

➔ **Проверка аргументов:**

- `verify(mockObject).methodName(exactArgument);` - точное совпадение аргумента.
- `verify(mockObject).methodName(anyString());` - проверка с использованием матчеров (например, `Mockito.any()`).

➔ **Порядок вызовов:** Используется `InOrder` для проверки последовательности действий.

6. ТЕСТИРОВАНИЕ В SPRING (SPRING BOOT TEST).

ОСНОВНАЯ КОНЦЕПЦИЯ — TEST SLICING (ТЕСТИРОВАНИЕ «СРЕЗАМИ»). ВМЕСТО ТОГО ЧТОБЫ КАЖДЫЙ РАЗ ЗАГРУЖАТЬ ВСЁ ПРИЛОЖЕНИЕ ЦЕЛИКОМ, ВЫ МОЖЕТЕ ИНИЦИАЛИЗИРОВАТЬ ТОЛЬКО НУЖНЫЕ СЛОИ (БИНЫ), ЧТО ЗНАЧИТЕЛЬНО УСКОРЯЕТ ВЫПОЛНЕНИЕ.

ОСНОВНЫЕ АННОТАЦИИ «СРЕЗОВ» :

- ➔ **@WebMvcTest:** Тестирует только веб-слой (контроллеры).
 - Загружает компоненты **Spring MVC**, но не загружает сервисы или репозитории.
 - Используется вместе с **MockMvc** для имитации **HTTP-запросов**.
- ➔ **@DataJpaTest:** Используется для тестирования репозитория (уровня данных).
 - Автоматически настраивает базу данных в памяти (например, **H2**).
 - Включает **логирование SQL-запросов** и по умолчанию откатывает транзакции после каждого теста.
- ➔ **@JsonTest:** Проверяет правильность сериализации и десериализации JSON (библиотеки **Jackson/Gson**).

ПОЛНОЕ ИНТЕГРАЦИОННОЕ ТЕСТИРОВАНИЕ.

@SPRINGBOOTTEST: ЗАПУСКАЕТ ПОЛНЫЙ КОНТЕКСТ ПРИЛОЖЕНИЯ.

- Используется для **сквозных тестов**, когда нужно проверить взаимодействие всех слоев.
- Параметр **webEnvironment** позволяет запустить реальный сервер на случайном порту (**RANDOM_PORT**).

ИНСТРУМЕНТЫ ДЛЯ МОКИРОВАНИЯ (ЗАГЛУШЕК):

- **@MockBean**: Создает **Mock-объект** и добавляет его в контекст Spring, заменяя существующий бин. Удобно, когда в **@WebMvcTest** нужно «подменить» реальный сервис.
- **@SpyBean**: Позволяет обернуть реальный бин из контекста в шпиона (spy), чтобы отслеживать вызовы или подменять только часть методов.

7. ЧТО ТАКОЕ TESTCONTAINERS? —
ЭТО БИБЛИОТЕКА НА БАЗЕ DOCKER,
ПОЗВОЛЯЮЩАЯ ЗАПУСКАТЬ
РЕАЛЬНЫЕ ЭКЗЕМПЛЯРЫ БД
(POSTGRESQL, REDIS) ИЛИ БРОКЕРОВ
СООБЩЕНИЙ (КАФКА) ПРЯМО ИЗ КОДА
ТЕСТА. ЭТО ИЗБАВЛЯЕТ ОТ ПРОБЛЕМ
С РАЗЛИЧИЯМИ МЕЖДУ H2 И
РЕАЛЬНОЙ БД.

XX.

GIT REPOSITORY И ВСЯКИЕ КОМАНДЫ.

**1. ЧТО ТАКОЕ GIT? — ЭТО БЕСПЛАТНАЯ
РАСПРЕДЕЛЕННАЯ СИСТЕМА
КОНТРОЛЯ ВЕРСИЙ, КОТОРАЯ
ПОЗВОЛЯЕТ ОТСЛЕЖИВАТЬ
ИЗМЕНЕНИЯ В ФАЙЛАХ, СОХРАНЯТЬ
ИСТОРИЮ ПРОЕКТА И ЭФФЕКТИВНО
РАБОТАТЬ НАД КОДОМ КАК В
ОДИНОЧКУ, ТАК И В КОМАНДЕ.**

2. ЗАЧЕМ НУЖЕН GIT И КАКИЕ СУЩЕСТВУЮТ КОМАНДЫ?

- **История изменений:** позволяет вернуться к любой сохраненной точке (коммиту), если в коде что-то сломалось.
- **Командная работа:** программисты могут одновременно редактировать одни и те же файлы без риска перезаписать чужую работу.
- **Безопасные эксперименты:** новые функции разрабатываются в изолированных «ветках», что защищает рабочий код от случайных ошибок.

СТАРТ ПРОЕКТА (ИНИЦИАЛИЗАЦИЯ)

- **git init** - создает новый пустой Git-репозиторий в текущей папке.
- **git clone <ссылка>** - скачивает существующий проект с сервера (GitHub/GitLab) на компьютер.

ЕЖЕДНЕВНАЯ РАБОТА (СОХРАНЕНИЕ ИЗМЕНЕНИЙ)

- **git status** - показывает состояние файлов: что изменилось, что готово к сохранению.
- **git diff** - показывает конкретные измененные строки кода до их сохранения.
- **git add <файл>** - добавляет конкретный файл в индекс (подготавливает к сохранению).
- **git add .** - добавляет все измененные и новые файлы в индекс.
- **git commit -m "сообщение"** - создает «снимок» (коммит) изменений с описанием того, что было сделано.

РАБОТА С ВЕТКАМИ (ПАРАЛЛЕЛЬНАЯ РАЗРАБОТКА)

- **git branch** - показывает список всех локальных веток в проекте.
- **git branch <название>** - создает новую ветку для новой задачи или эксперимента.
- **git checkout <название>** - переключает вас на указанную ветку.
- **git switch <название>** - современная и более удобная замена для переключения веток.
- **git merge <название>** - вливает изменения из указанной ветки в ту, в которой вы сейчас находитесь.
- **git merge —abort** - «Экстренная отмена»: останавливает процесс слияния веток, если в процессе возникли конфликты, и возвращает ваш проект в то состояние, в котором он был до запуска команды **git merge** или **git pull**.
- **git branch -d <название>** - удаляет ветку (обычно после того, как её код успешно объединили с основной).
- **git stash** - «прячет» все текущие незакоммиченные изменения. Рабочая директория становится чистой.
- **git stash pop** - возвращает спрятанные файлы обратно, когда вы вернулись к работе.

ПРОСМОТР ИСТОРИИ И ОТМЕНА

- **git log** - выводит историю всех коммитов (автор, дата, описание).
- **git log --oneline** - показывает историю в кратком и компактном виде (один коммит — одна строка).
- **git checkout <хэш_коммита>** - позволяет временно «вернуться в прошлое» и посмотреть код на момент этого коммита.
- **git reset --hard <хэш_коммита>** - безвозвратно откатывает проект к указанному коммиту, удаляя всё, что было сделано после него.
- **git reset --soft <хэш_коммита>** - коммит удалится, но весь ваш код останется в файлах как незакоммиченный.
- **git revert <хэш_коммита>** - создает новый коммит, который отменяет изменения старого (безопасный способ отмены).

СИНХРОНИЗАЦИЯ С СЕРВЕРОМ (GITHUB / GITLAB)

- **git remote add origin <ссылка>** - связывает ваш локальный репозиторий с удаленным сервером.
- **git push -u origin <ветка>** - отправляет вашу ветку и все коммиты на сервер (первый раз).
- **git push** - отправляет новые коммиты на сервер (в последующие разы).
- **git fetch** - проверяет сервер и скачивает информацию о новых изменениях, но не внедряет их в ваш код.
- **git pull** - скачивает изменения с сервера и сразу объединяет (мержит) их с вашим локальным кодом.

GIT REBASE - I ОБЪЕДИНЕНИЕ КОММИТОВ. ПОШАГОВЫЙ АЛГОРИТМ:

- **git rebase -i HEAD~3** (для 3 последних коммитов)
- В открывшемся редакторе оставить pick у верхнего коммита и заменить на squash (или s) у остальных. Сохранить и выйти из файла.
- В новом окне стереть все комментарии и написать одно новое. Сохранить и выйти из файла.

ЕСЛИ ВЫ ХОТИТЕ ОБЪЕДИНИТЬ КОММИТЫ НАЧИНАЯ С КАКОГО-ТО КОНКРЕТНОГО СТАРОГО, ВМЕСТО HEAD~3 (3 ПОСЛЕДНИХ КОММИТА) МОЖНО УКАЗАТЬ ХЭШ КОММИТА, КОТОРЫЙ ИДЕТ ПЕРЕД ТЕМИ, ЧТО НУЖНО ОБЪЕДИНИТЬ.

3. ЧТО ТАКОЕ CI/CD (CONTINUOUS INTEGRATION И CONTINUOUS DELIVERY / DEPLOYMENT)? — ЭТО АВТОМАТИЧЕСКИЙ КОНВЕЙЕР (PIPELINE), КОТОРЫЙ БЕРЕТ ВАШ КОД ИЗ GIT, ПРОВЕРЯЕТ ЕГО НА ОШИБКИ И САМ ВЫКЛАДЫВАЕТ НА РАБОТАЮЩИЙ САЙТ ИЛИ В ПРИЛОЖЕНИЕ.

- **CI (Непрерывная интеграция):** Автоматическая проверка кода (Сборка, Линтинг (code style), Тесты).
- **CD (Непрерывная доставка):** Автоматический деплой (Выкатка проверенного кода на реальный сайт/сервер).

4. ЧТО ТАКОЕ PULL REQUEST (PR) ИЛИ MERGE REQUEST / MR)? — ЭТО ЗАПРОС НА ОДОБРЕНИЕ ВАШИХ ИЗМЕНЕНИЙ ПЕРЕД ТЕМ, КАК ОНИ БУДУТ ВЛИТЫ В ОСНОВНУЮ ВЕТКУ ПРОЕКТА.

КАК ВЫГЛЯДИТ ПРОЦЕСС:

- **Создание ветки:** Вы создаете свою локальную ветку (например, `feature/login`) и пишете там код задачи.
- **Пуш на сервер:** Вы отправляете эту ветку на GitHub командой `git push origin feature/login`.
- **Открытие PR:** Вы заходите на сайт GitHub, нажимаете кнопку «**New pull request**», выбираете свою ветку и ветку, куда хотите влить код (обычно это `main` или `develop`).
- **Code Review (Проверка кода):** Коллеги открывают ваш PR, смотрят изменения по строчкам, оставляют комментарии, просят исправить ошибки при необходимости.
- **Тесты CI/CD:** Параллельно роботы автоматически запускают тесты.
- **Слияние (Merge):** Когда тесты пройдены (горит зеленая галочка), а коллеги поставили Approve, нажимается кнопка **Merge**, и ваш код официально становится частью главного проекта.

THE END.

